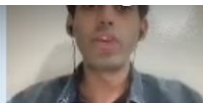


POD Attention

Slides from Aditya Kamath ASPLOS 2025 talk, Dinu Goyal
explanations



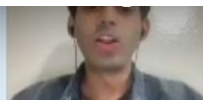
Outline



- LLM Inference and Hybrid Batching
- Motivation for POD-Attention
- GPU microarchitectural background and kernel fusion
- POD-Attention design
- Evaluation



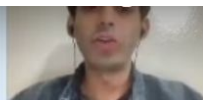
LLM Prefill



Large Language Model (LLM)



LLM Prefill



Self-Attention

Large Language Model (LLM)



LLM Prefill



N Layers X

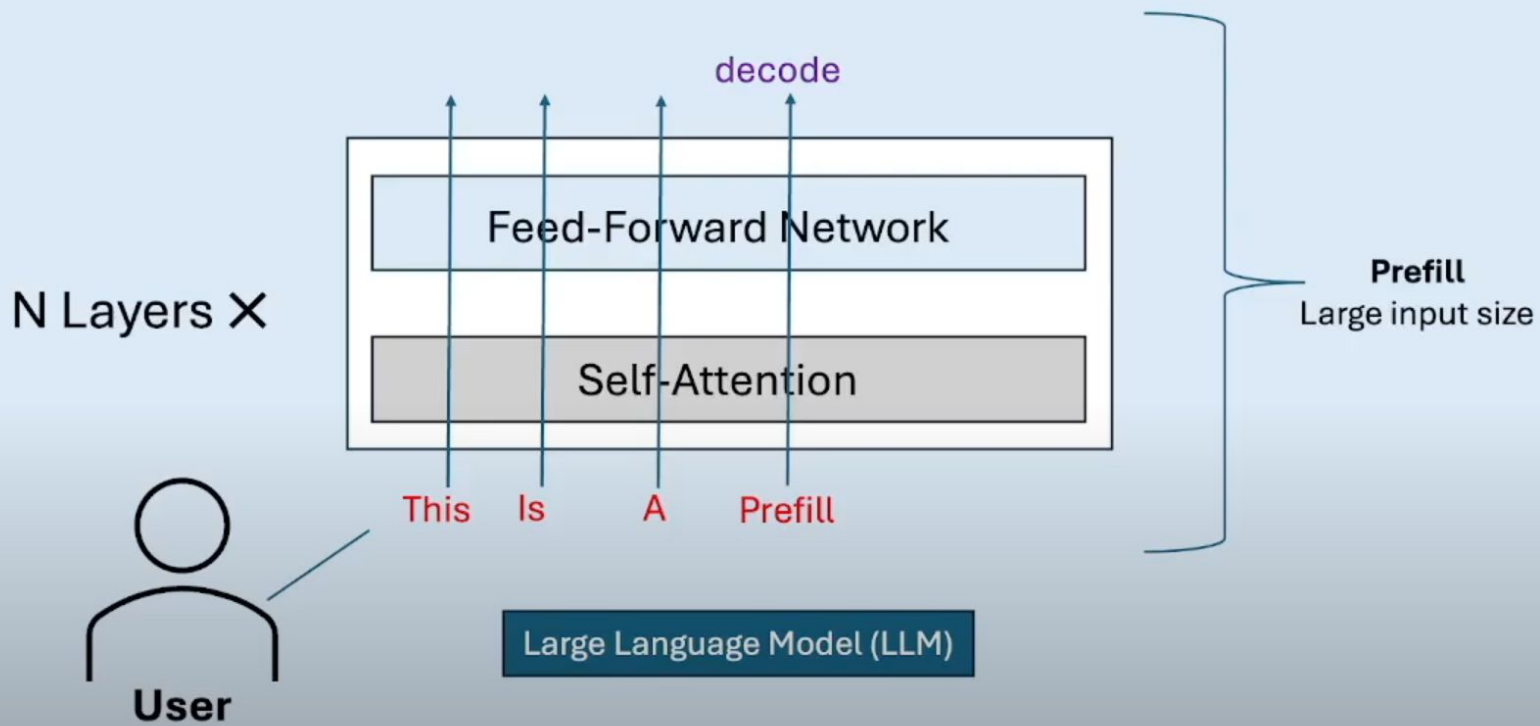
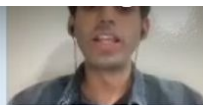
Feed-Forward Network

Self-Attention

Large Language Model (LLM)

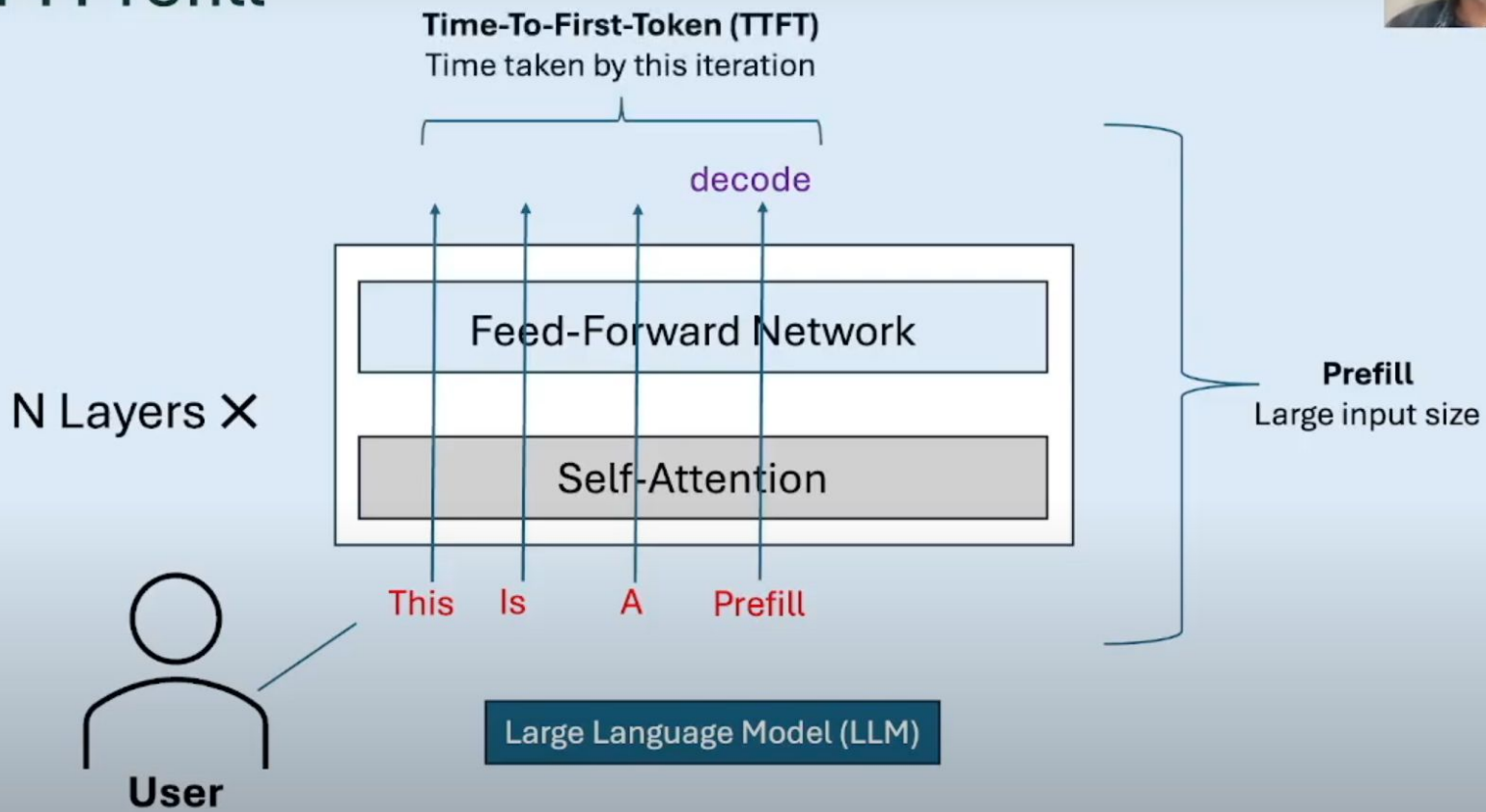
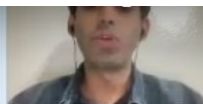


LLM Prefill



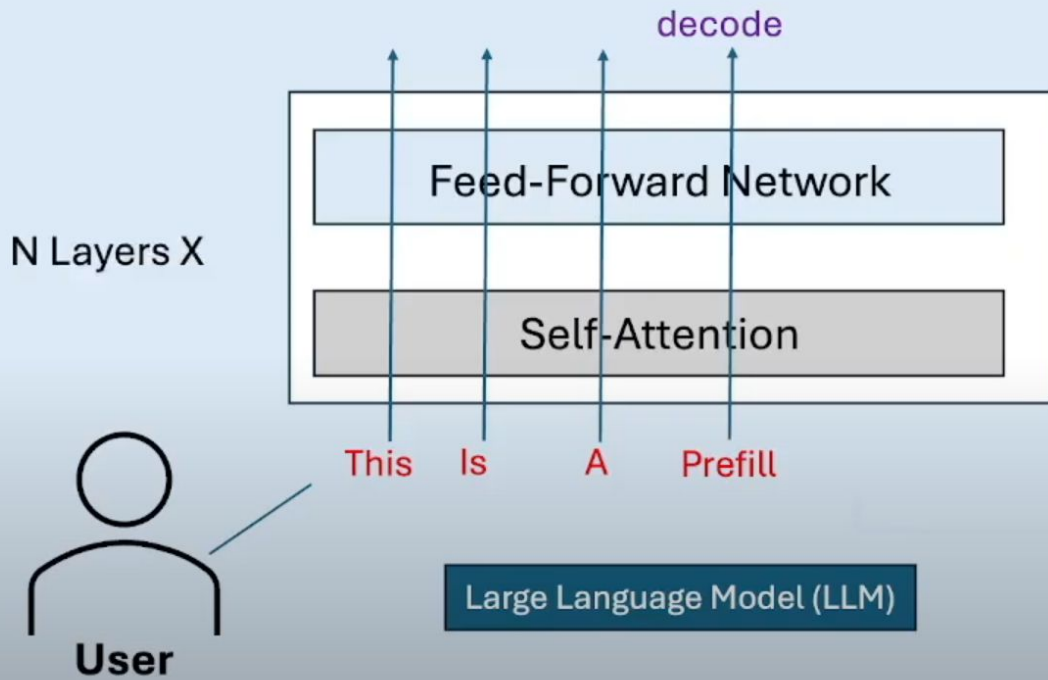
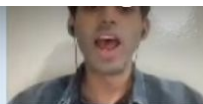


LLM Prefill



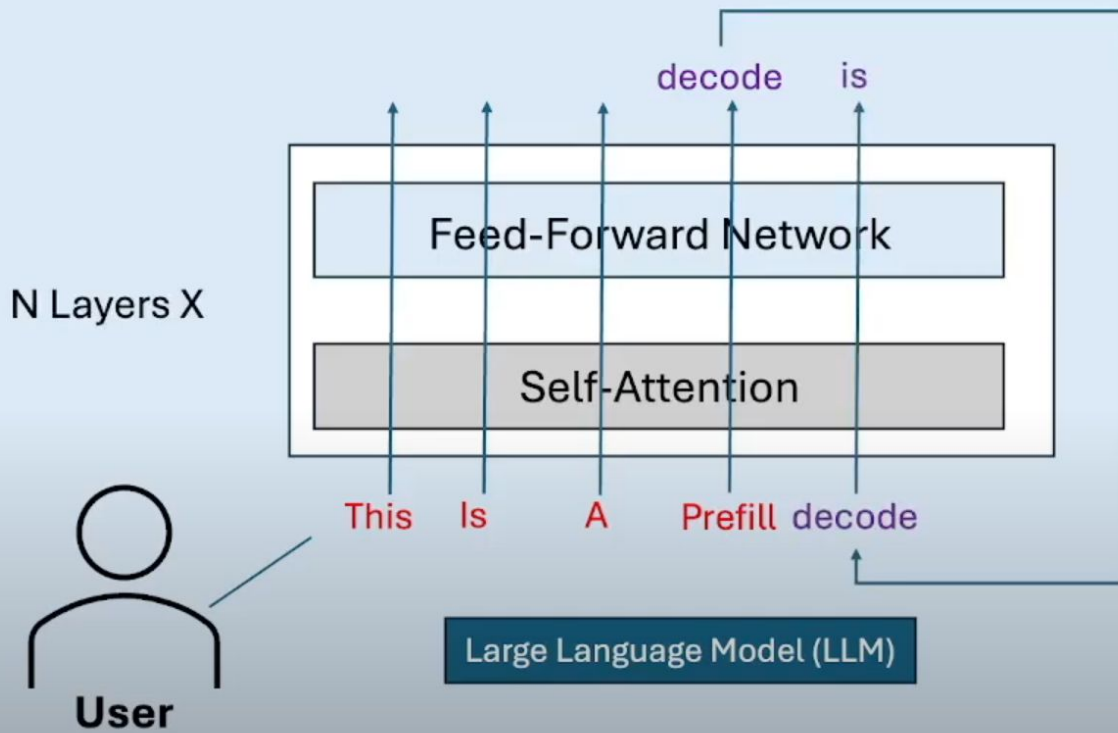
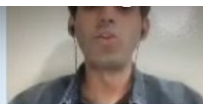


LLM Decode



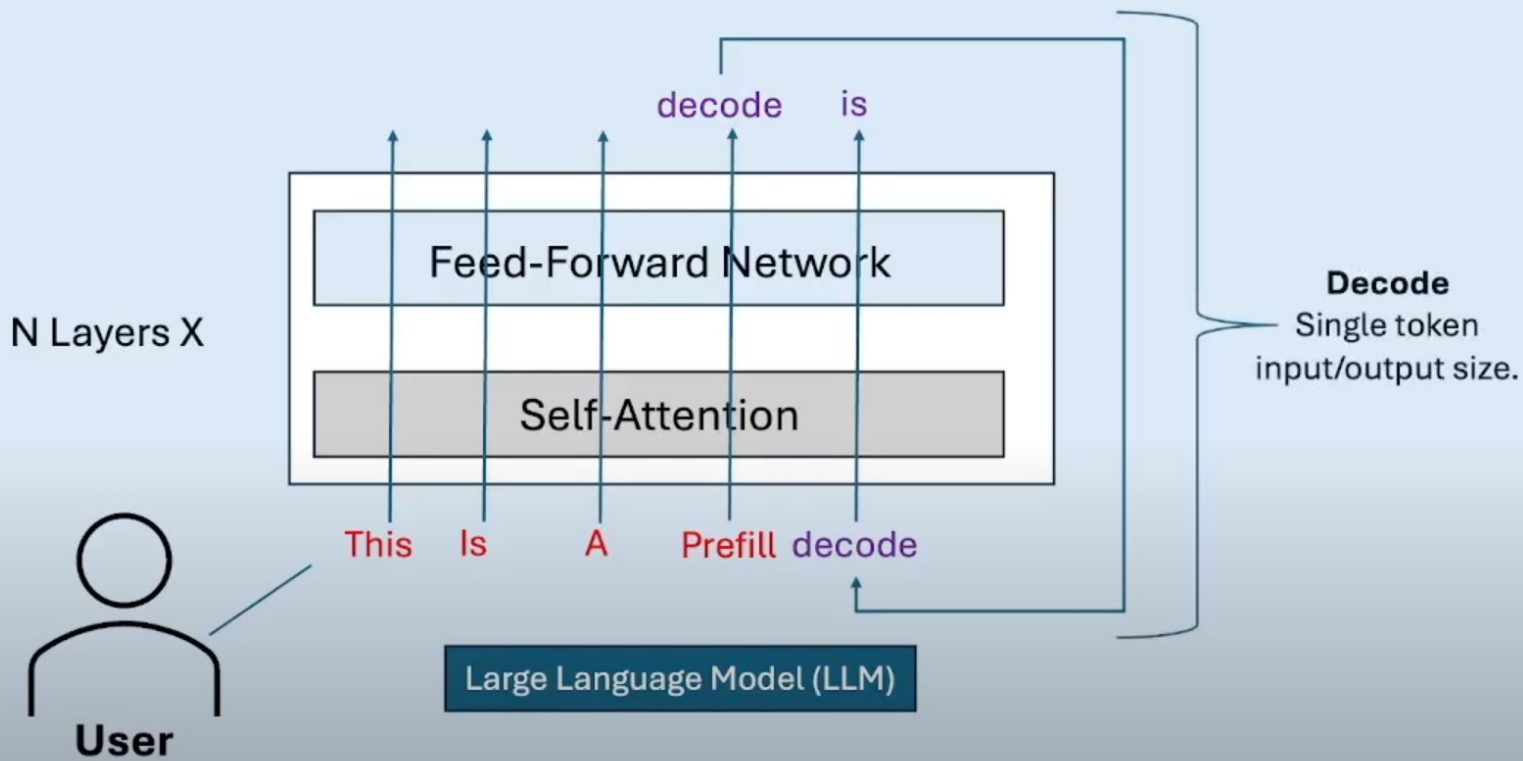
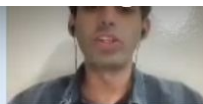


LLM Decode



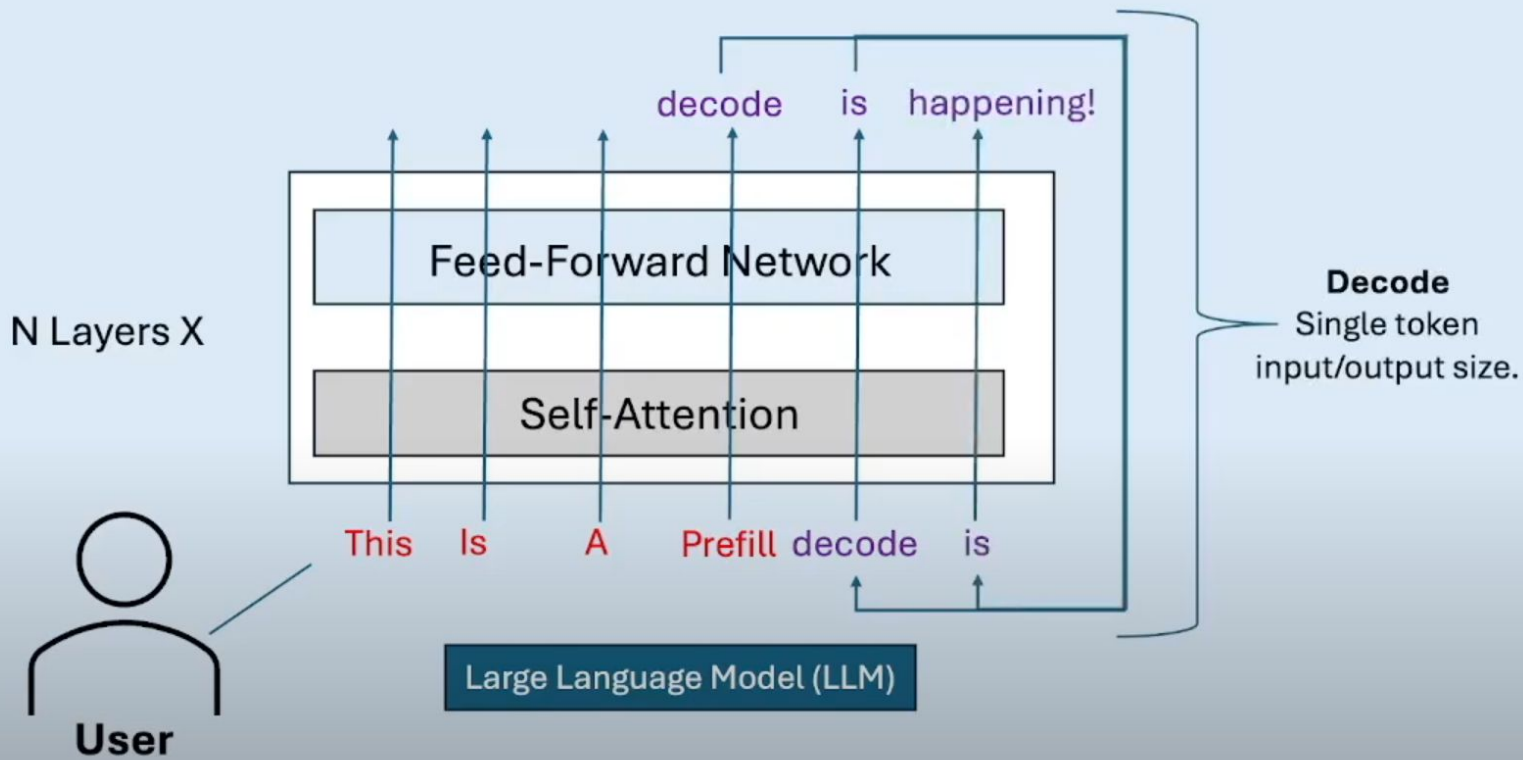
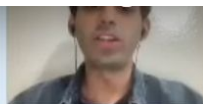


LLM Decode



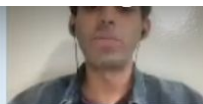


LLM Decode

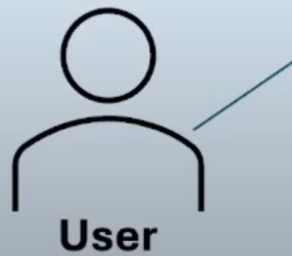




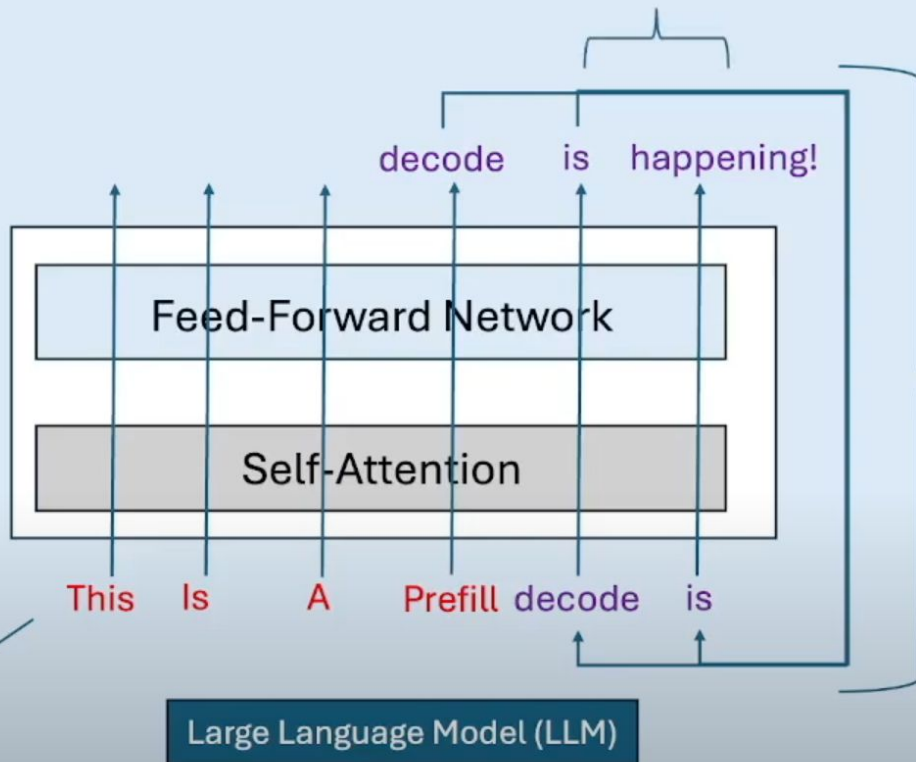
LLM Decode



N Layers X



User



Time-Between-Token (TBT)

Time between two decodes

decode is happening!

Feed-Forward Network

Self-Attention

This Is A Prefill decode is

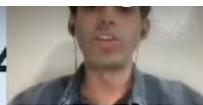
Large Language Model (LLM)

Decode

Single token
input/output size.

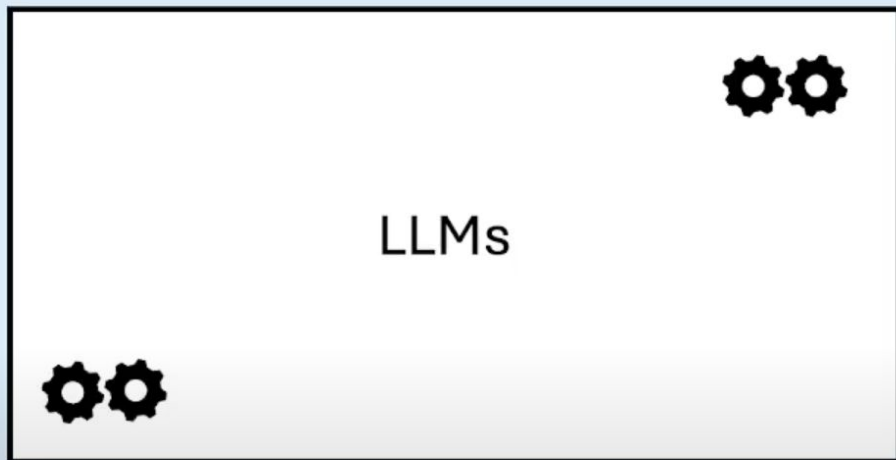


LLM Hybrid Batching [Sarathi-Serve, OSDI'24]



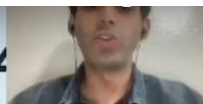
Prefill

Decode





LLM Hybrid Batching [Sarathi-Serve, OSDI'24]

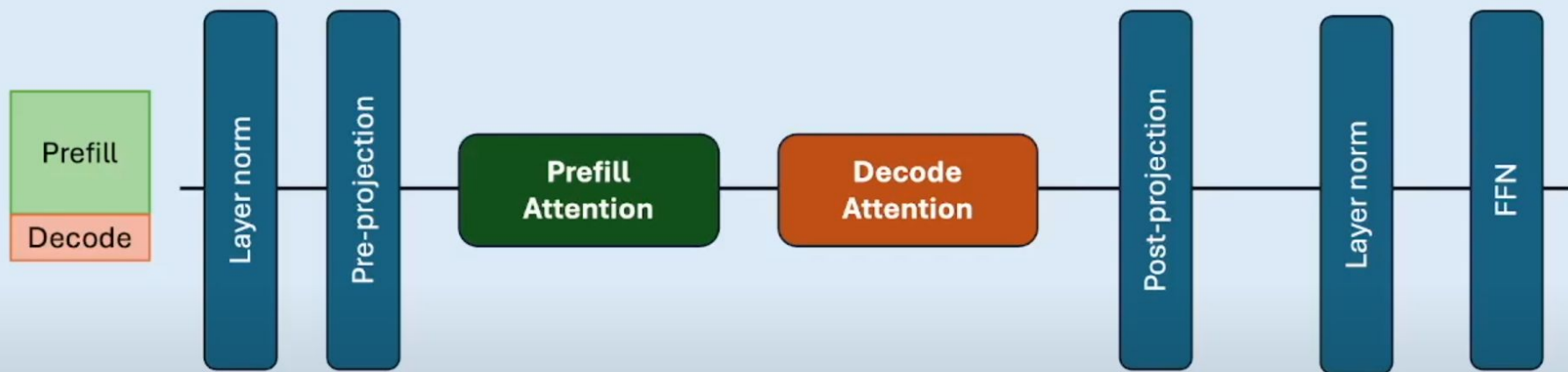
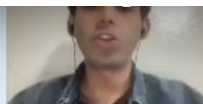


Prefill and decode inputs of multiple requests are combined as a single input

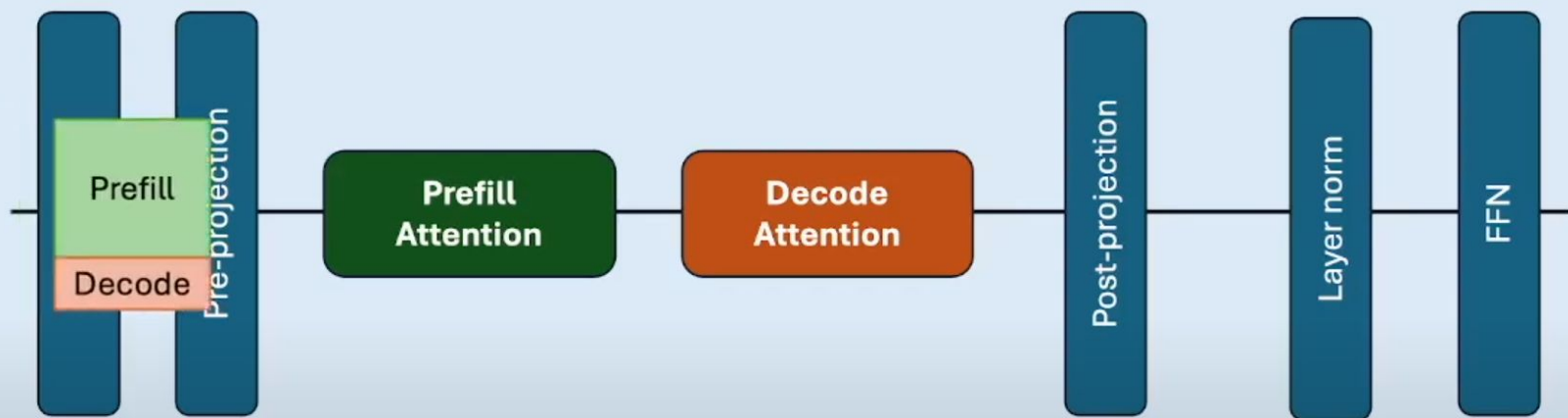
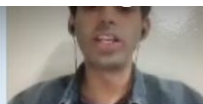


Improves throughput by reusing model weights and reducing latencies.

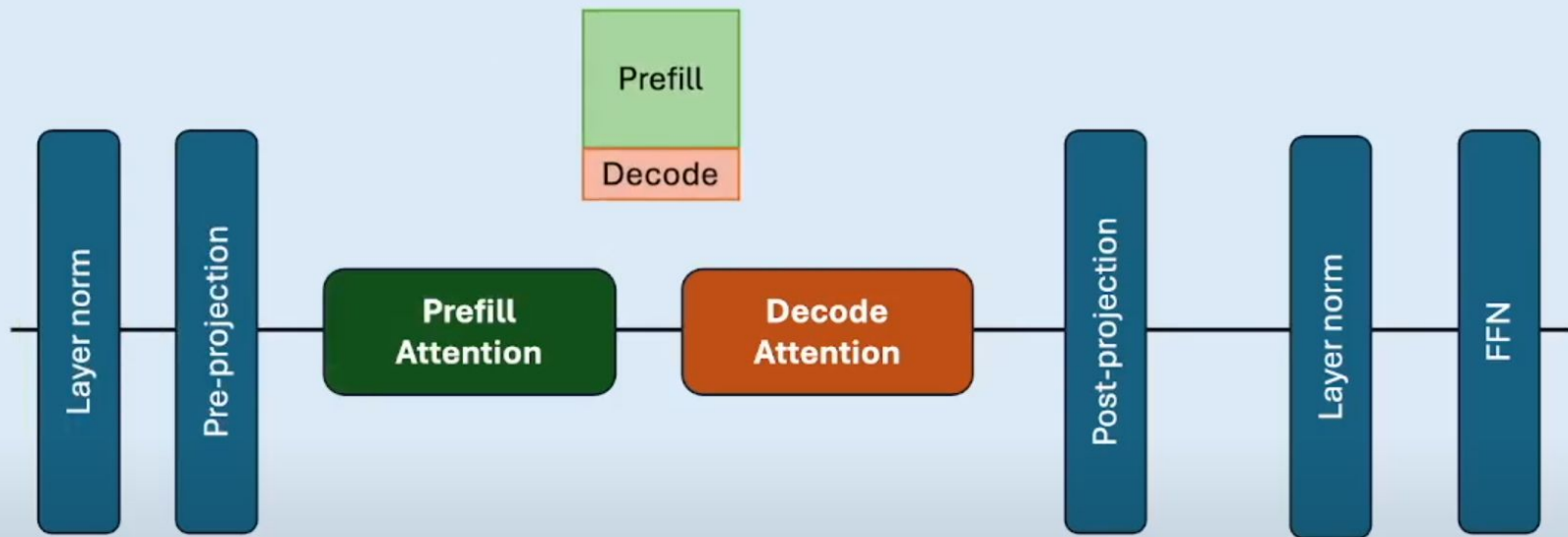
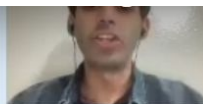
LLM Hybrid Batching – A Closer Look



LLM Hybrid Batching – A Closer Look

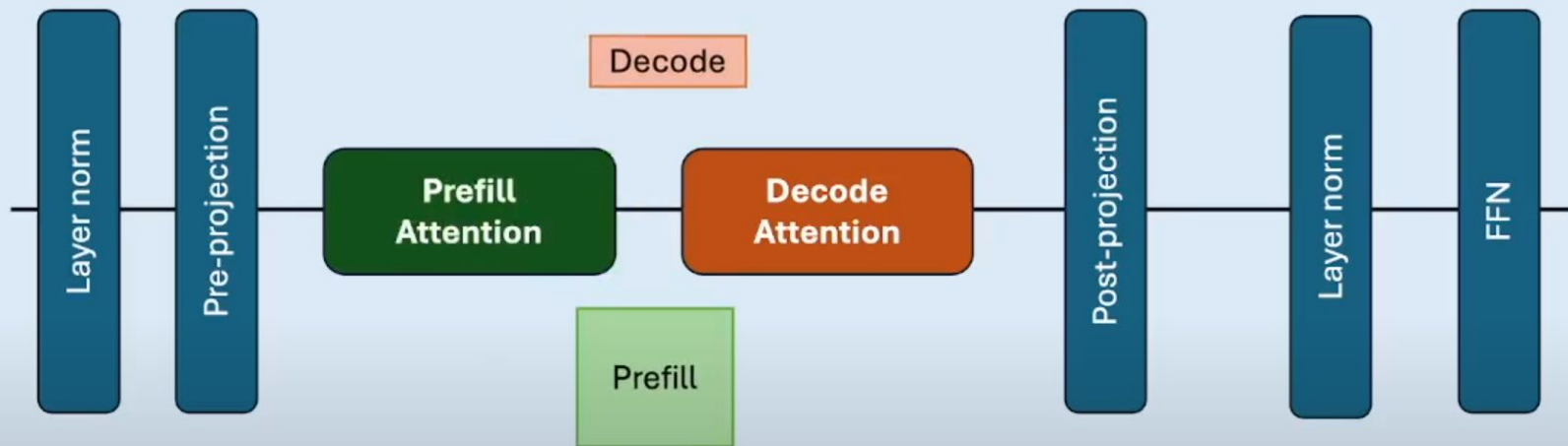
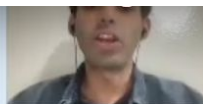


LLM Hybrid Batching – A Closer Look



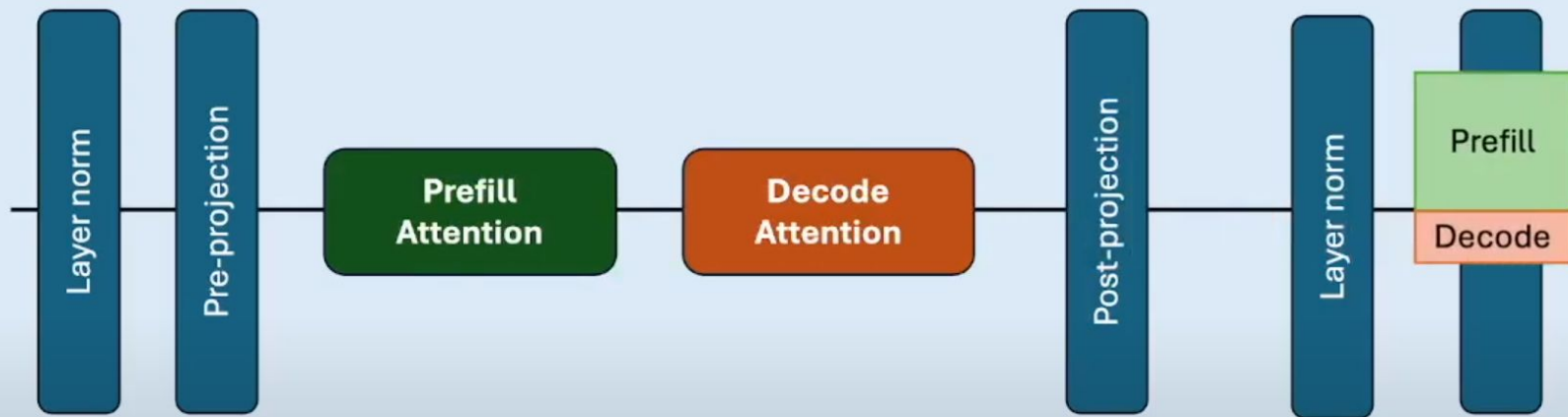
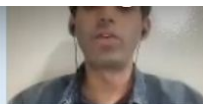
Attention operates on discrete data items (no data reuse across input requests).
Kernels optimized independently for prefill or decode.

LLM Hybrid Batching – A Closer Look



Attention operates on discrete data items (no data reuse across input requests).
Kernels optimized independently for prefill or decode.

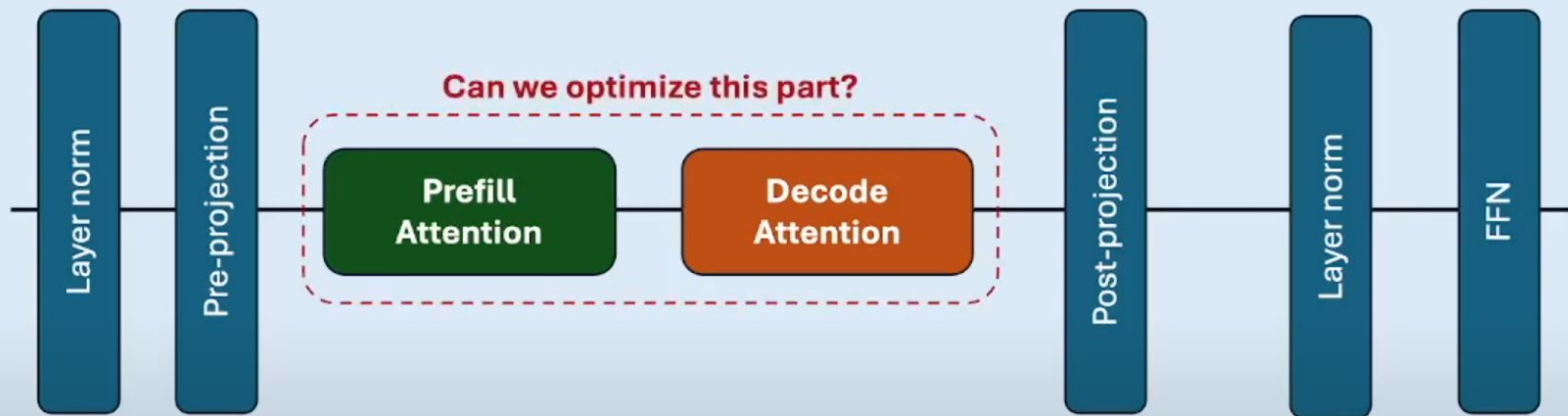
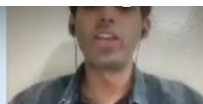
LLM Hybrid Batching – A Closer Look



Attention operates on discrete data items (no data reuse across input requests).
Kernels optimized independently for prefill or decode.



Our Goal With POD-Attention



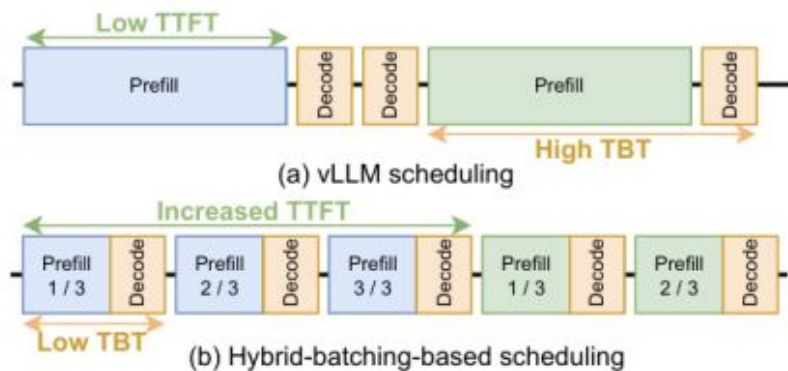


Figure 2. Impact of scheduling strategies on TTFT and TBT.

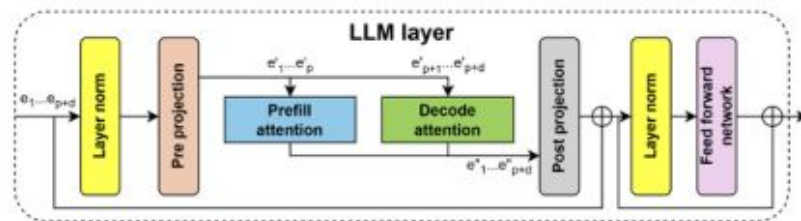


Figure 3. Computation in hybrid batches. Current systems compute prefill inputs ($e_1 \dots e_p$) and decode inputs ($e_{p+1} \dots e_{p+d}$) together for linear operations. However, they compute prefill and decode attention separately using specialized kernels.

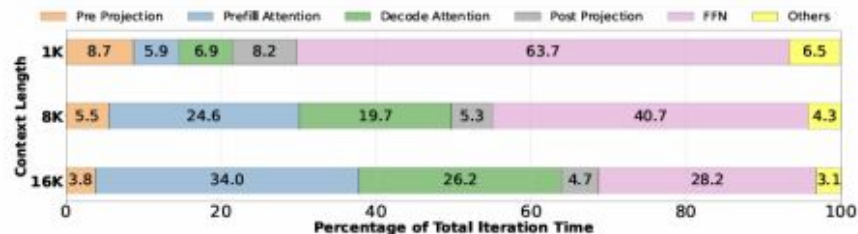
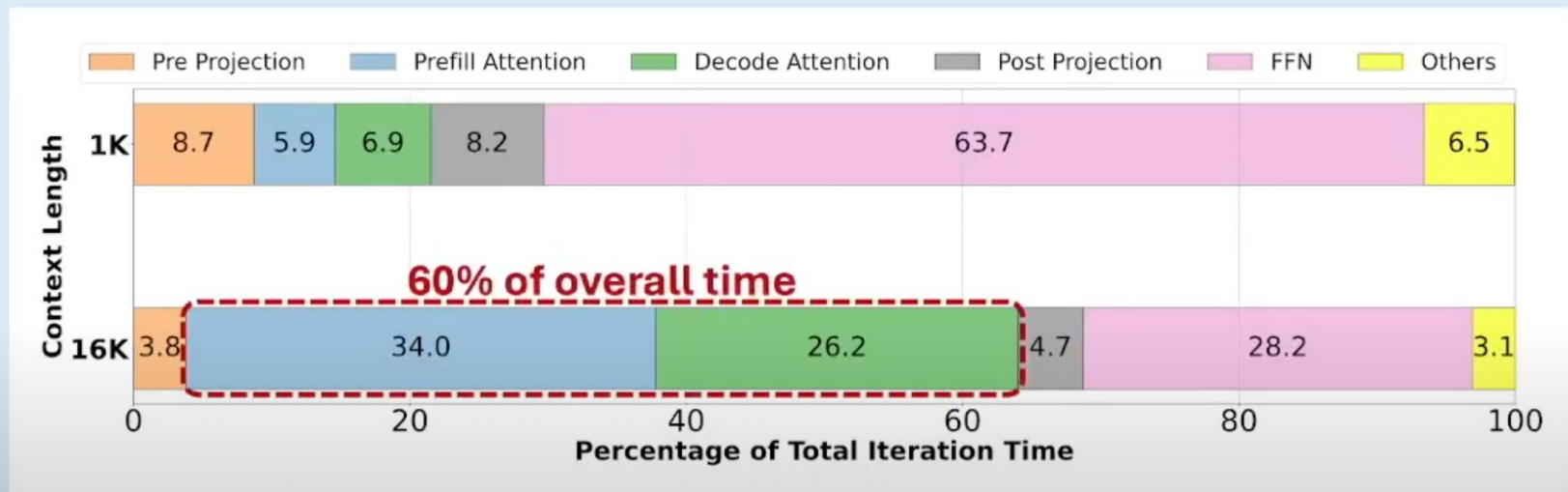
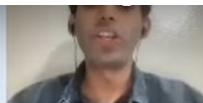


Figure 4. Contribution of different operations in iteration runtime with hybrid batching (model: Llama-3-8B, batch size: 60, chunk size: 1K). For each context length, we show runtime of iteration that processes the last chunk of a prompt.



Why Optimize Attention?

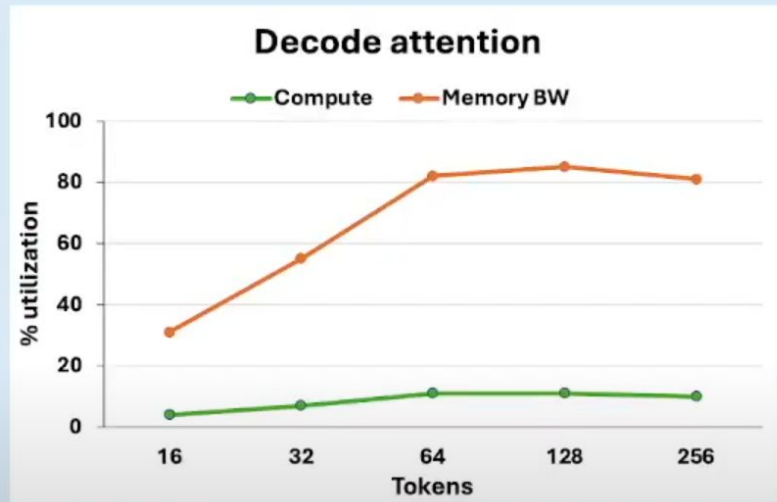
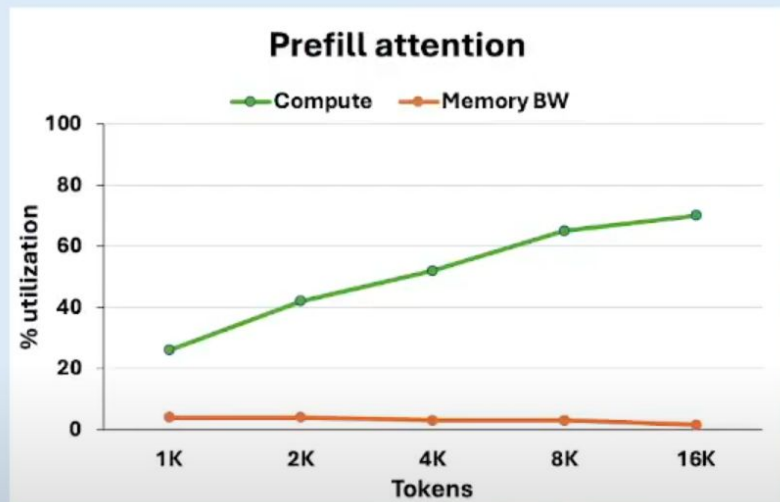
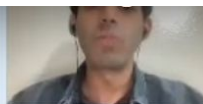


Model: Llama-3-8B, Hardware A100 x 2, Decode Batch Size: 60

Attention is the biggest bottleneck at large context lengths.



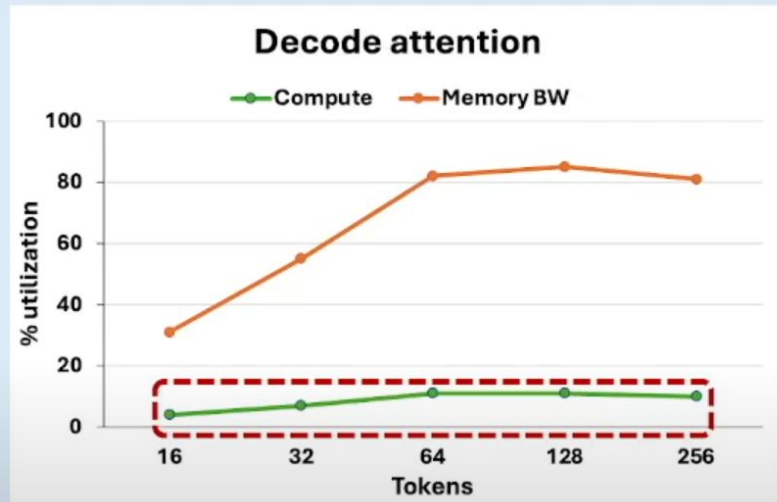
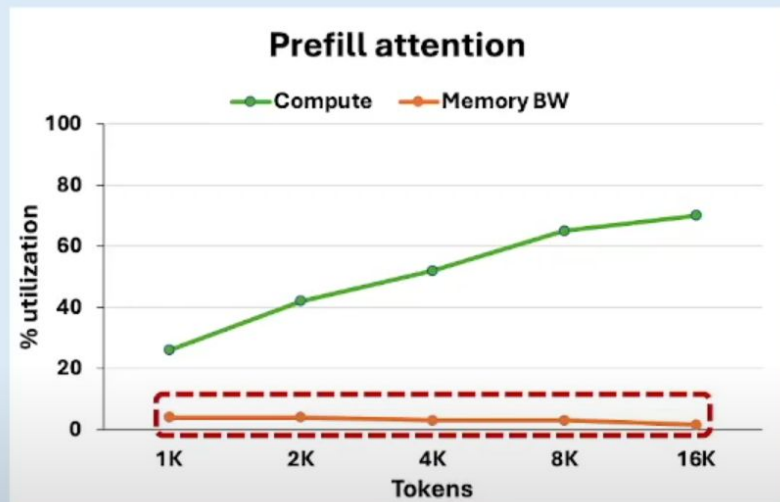
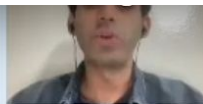
Opportunity: Resource (Under)utilization



Model: Llama-3-8B, Hardware: 2 A100



Opportunity: Resource (Under)utilization

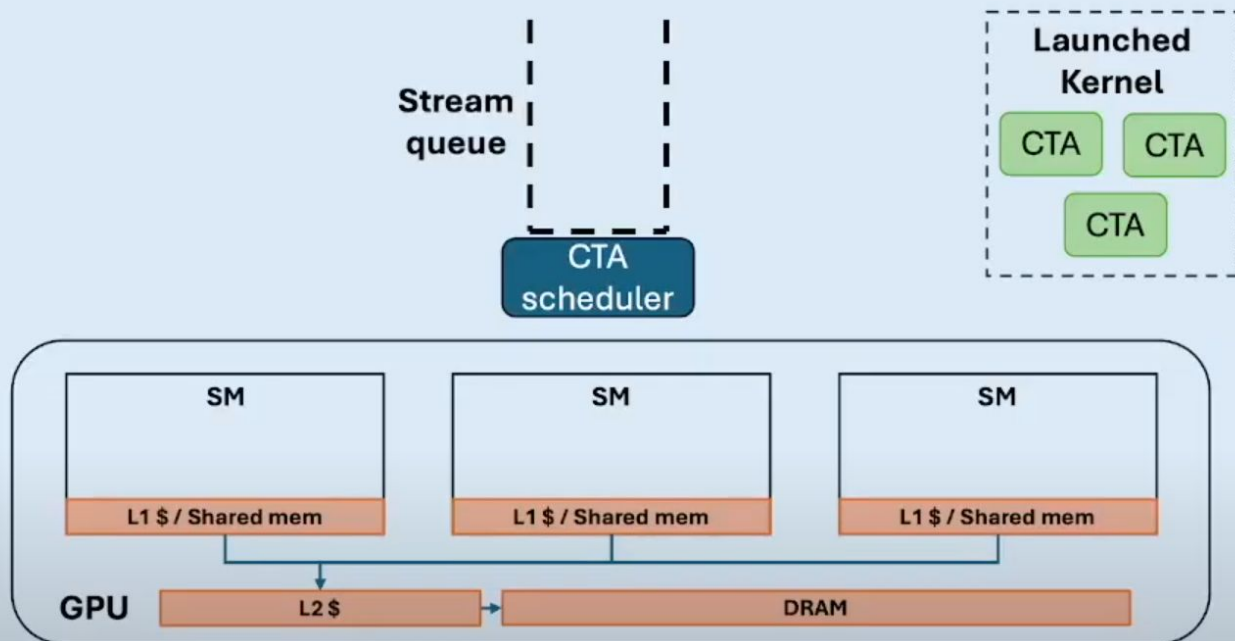
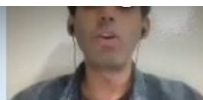


Model: Llama-3-8B, Hardware: 2 A100

Prefill saturates compute but has low memory bandwidth utilization.
Decode saturates memory bandwidth but has low compute utilization.



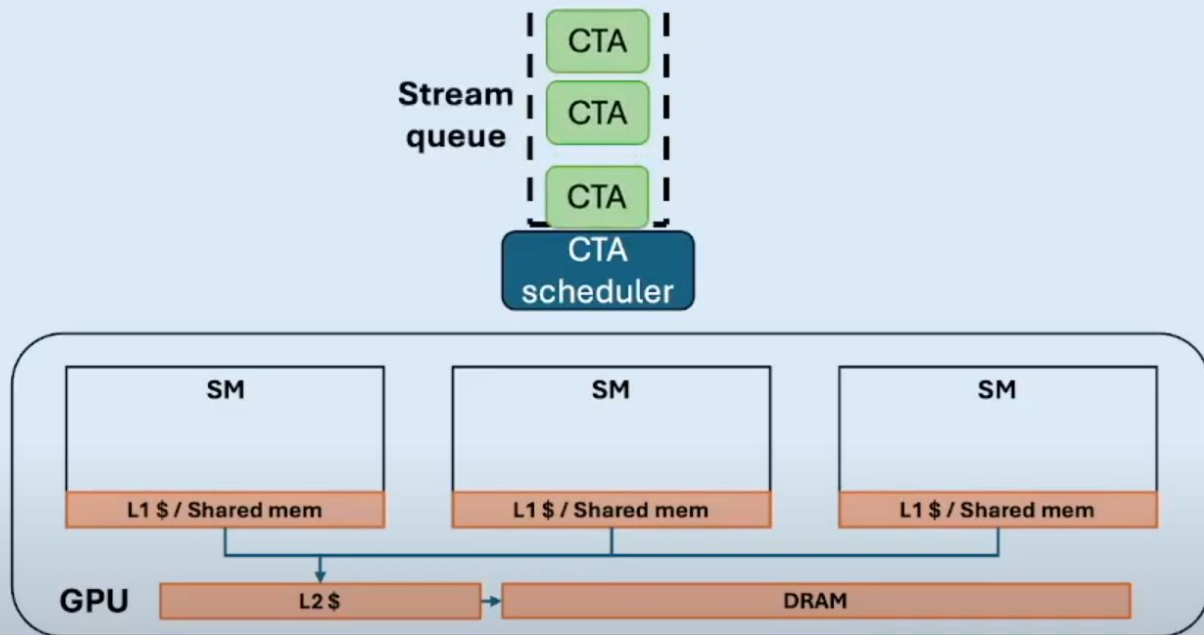
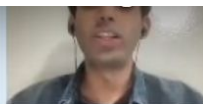
GPU Execution: CTA Scheduling



CTA scheduler in hardware assigns queued CTAs to SMs for execution.



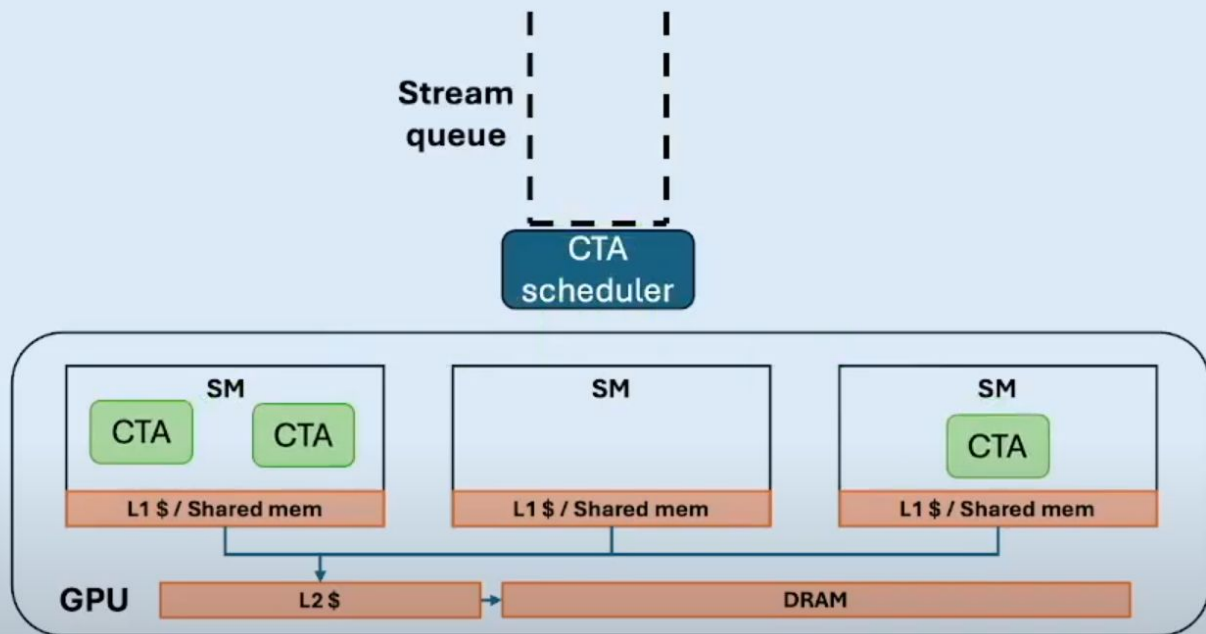
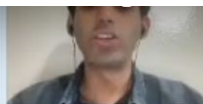
GPU Execution: CTA Scheduling



CTA scheduler in hardware assigns queued CTAs to SMs for execution.



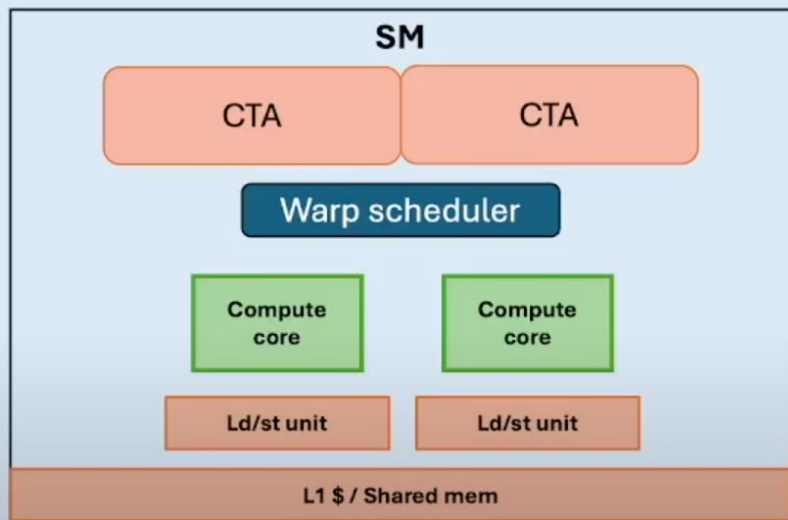
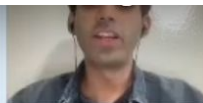
GPU Execution: CTA Scheduling



CTA scheduler in hardware assigns queued CTAs to SMs for execution.



GPU Execution: Warp Scheduling

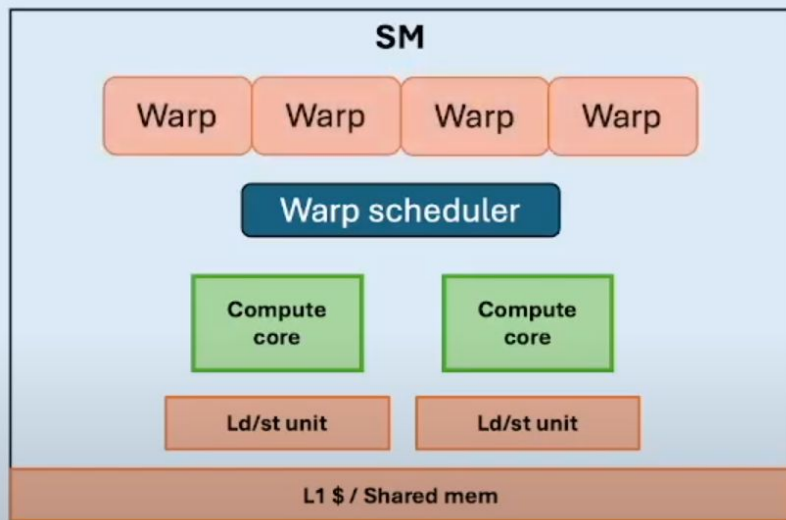
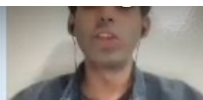


Green = Prefill Warp
Orange = Decode Warp

Warp scheduler assigns queued warps to free resources attempting to saturate all GPU resources.



GPU Execution: Warp Scheduling

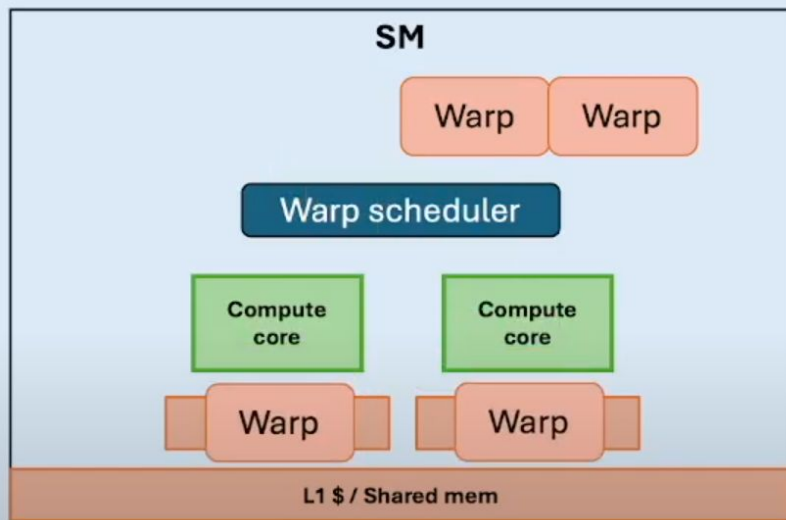
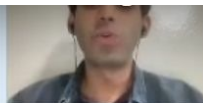


Green = Prefill Warp
Orange = Decode Warp

Warp scheduler assigns queued warps to free resources attempting to saturate all GPU resources.



GPU Execution: Warp Scheduling

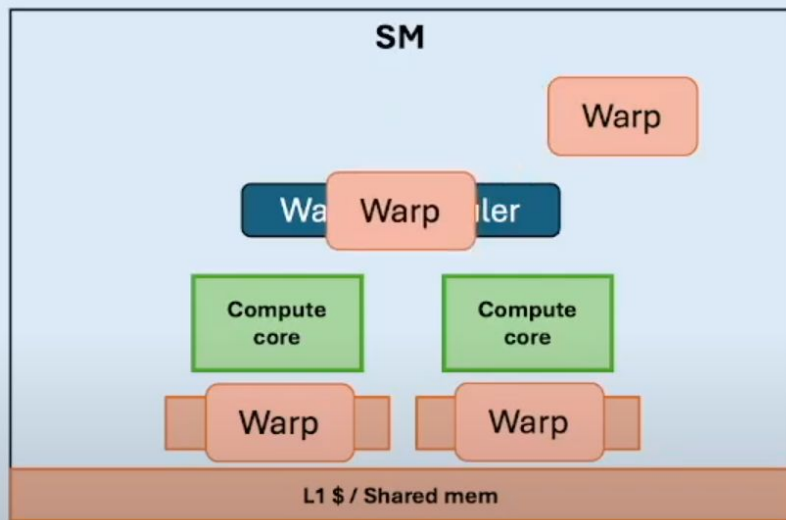
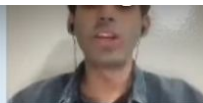


Green = Prefill Warp
Orange = Decode Warp

Warp scheduler assigns queued warps to free resources attempting to saturate all GPU resources.



GPU Execution: Warp Scheduling

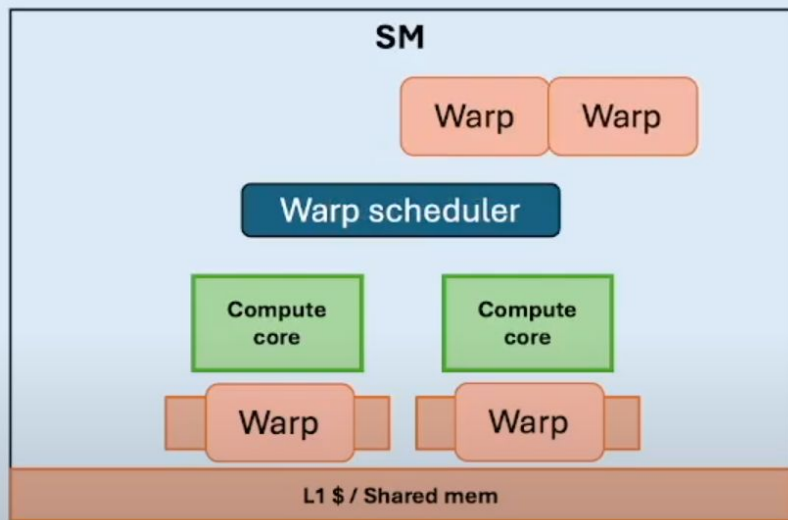
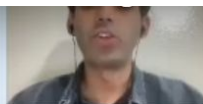


Green = Prefill Warp
Orange = Decode Warp

Warp scheduler assigns queued warps to free resources attempting to saturate all GPU resources.



GPU Execution: Warp Scheduling

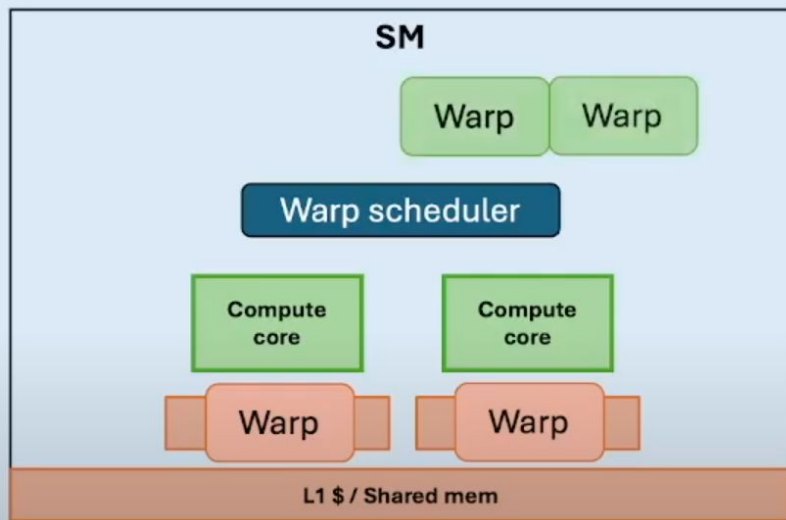
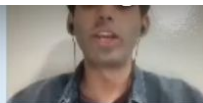


Green = Prefill Warp
Orange = Decode Warp

Warp scheduler assigns queued warps to free resources attempting to saturate all GPU resources.



GPU Execution: Warp Scheduling

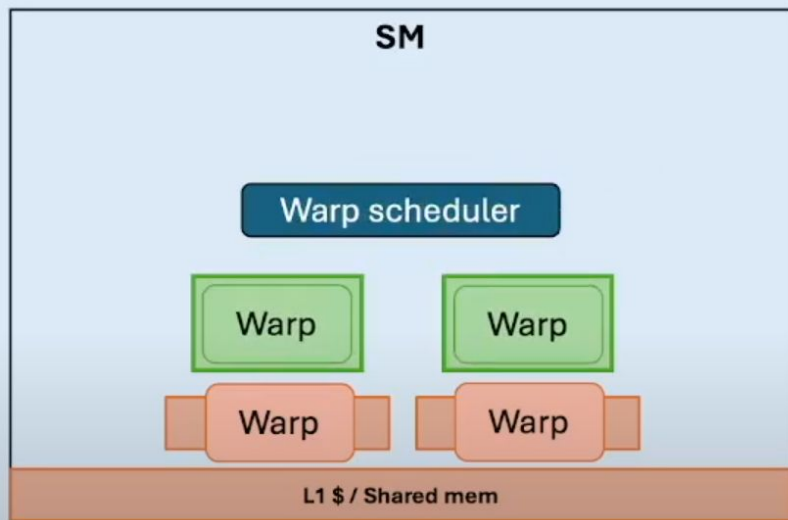
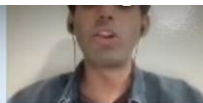


Green = Prefill Warp
Orange = Decode Warp

Warp scheduler assigns queued warps to free resources attempting to saturate all GPU resources.



GPU Execution: Warp Scheduling

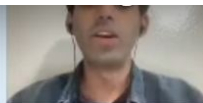


Green = Prefill Warp
Orange = Decode Warp

Warp scheduler assigns queued warps to free resources attempting to saturate all GPU resources.



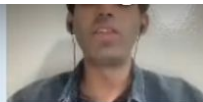
Fusing Prefill and Decode



- **Goal:** Overlap compute-heavy prefill with memory-bandwidth-heavy decode to fully utilize GPU resources.
 - **Sub Goal:** Co locate prefill and decode CTAs within the same SM



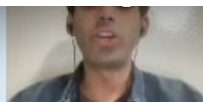
Known Methods of Kernel Fusion



- **Naïve fusion (CTA-parallel):** Combined kernel for both prefill and decode.
- **Kernel-parallel:** Launch kernels in separate CUDA Streams.
- **Warp-parallel:** Divide warps in the CTA among ops [HFuse, CGO '22].
- **Intra-thread:** Every thread performs both operations.



Known Methods of Kernel Fusion



- **Naïve fusion (CTA-parallel):** Combined kernel for both prefill and decode.
- **Kernel-parallel:** Launch kernels in separate CUDA Streams.

Cannot guarantee operation co-location within SM.

- **Warp-parallel:** Divide warps in the CTA among ops [HFuse, CGO '22].

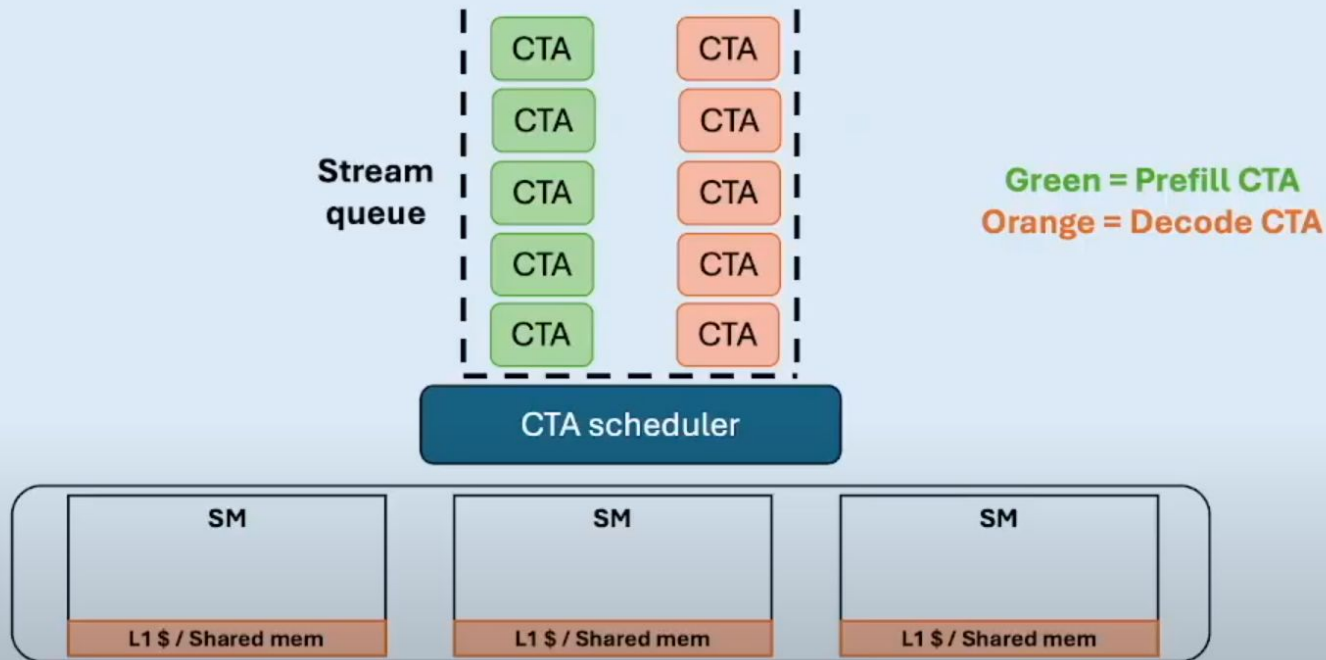
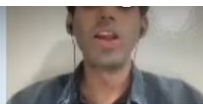
Exhibits bad load balancing within the GPU.

- **Intra-thread:** Every thread performs both operations.

Excessive synchronization overhead.



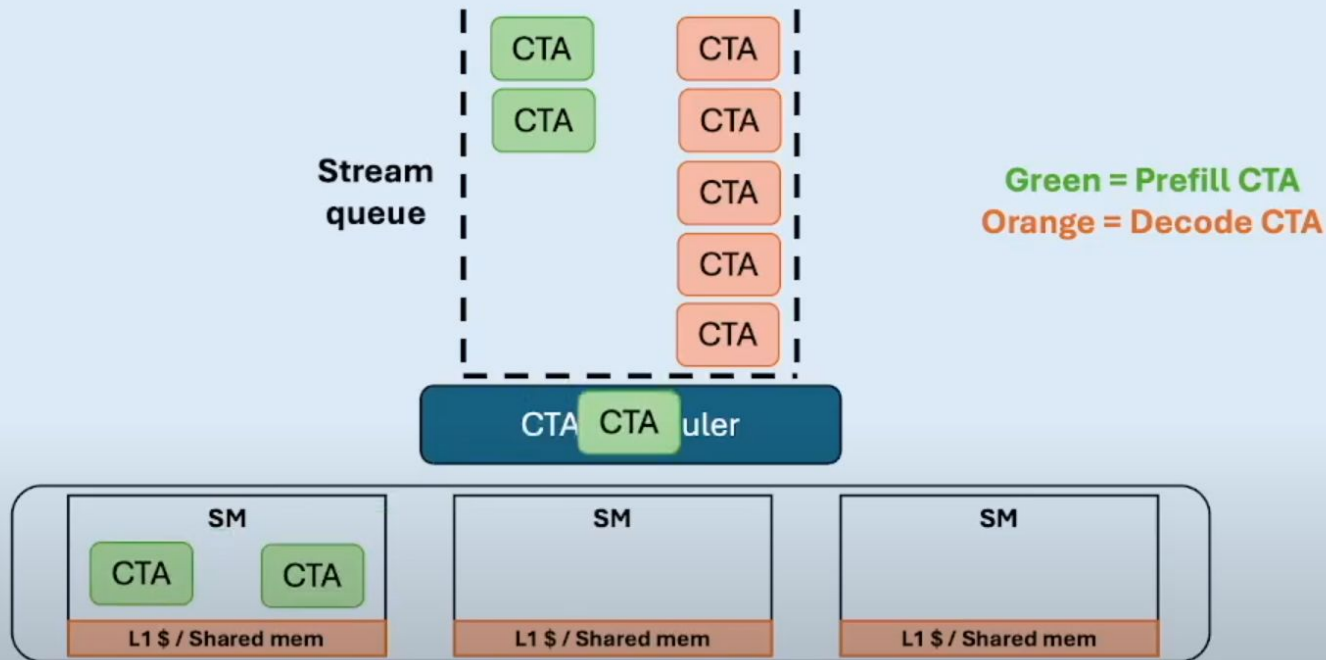
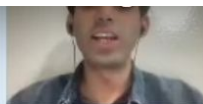
Naïve Kernel Fusion (CTA-Parallel)



Cannot guarantee SM co-location, limiting speedup.



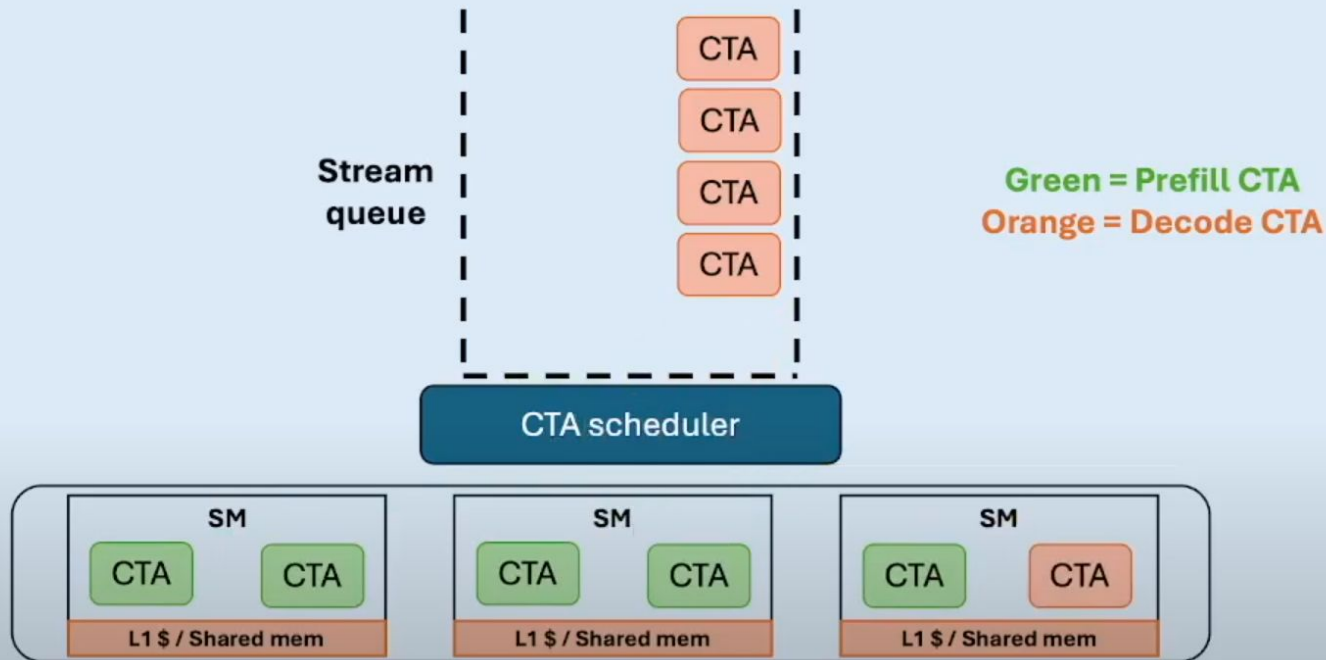
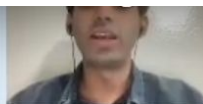
Naïve Kernel Fusion (CTA-Parallel)



Cannot guarantee SM co-location, limiting speedup.



Naïve Kernel Fusion (CTA-Parallel)



Cannot guarantee SM co-location, limiting speedup.

Execution method	GC	WQ	Notes
Streams [45]	×	✓	Easiest to implement
CTA	×	✓	Easy load balancing
Warp (e.g., HFuse [42])	✓	×	Suffers from straggler problem
Intra-thread [53, 59]	✓	×	Cannot overlap with CTA barriers
SM-aware CTA (Ours)	✓	✓	Minimizes operation interference

Table 2. Methods of concurrently executing or fusing different operations along different levels of the GPU execution hierarchy (GC=guarantees op co-location, WQ=reduces wave quantization).

GOAL: To Fuse Compute Heavy, Prefill Kernel with memory heavy decode kernel, to optimize resource utilization of the GPU

Straggler's Problem: When a job is split into many parallel tasks (e.g., computing gradients across multiple GPUs or nodes), the overall job often **has to wait for the slowest task to complete** before proceeding to the next step (e.g., weight update or aggregating results).

Wave Quantization: If the number of CTAs in a kernel is not divisible by the number of GPU SMs, some of the SMs in the last Wave of scheduling remain idle. This phenomenon is called Wave Quantization.



Known Methods of Kernel Fusion



- **Naïve fusion (CTA-parallel):** Combined kernel for both prefill and decode.

- **Kernel-parallel:** Launch kernels in separate CUDA Streams.

Cannot guarantee operation co-location within SM.

- **Warp-parallel:** Divide warps in the CTA among ops [HFuse, CGO '22].

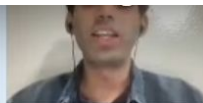
Exhibits bad load balancing within the GPU.

- **Intra-thread:** Every thread performs both operations.

Excessive synchronization overhead.



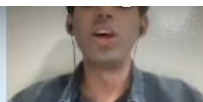
How Can We Address This?



- **Goal:** Overlap compute-heavy prefill with memory-bandwidth-heavy decode to fully utilize GPU resources.
- **Problem:** CTA scheduler is handled by GPU driver/hardware.



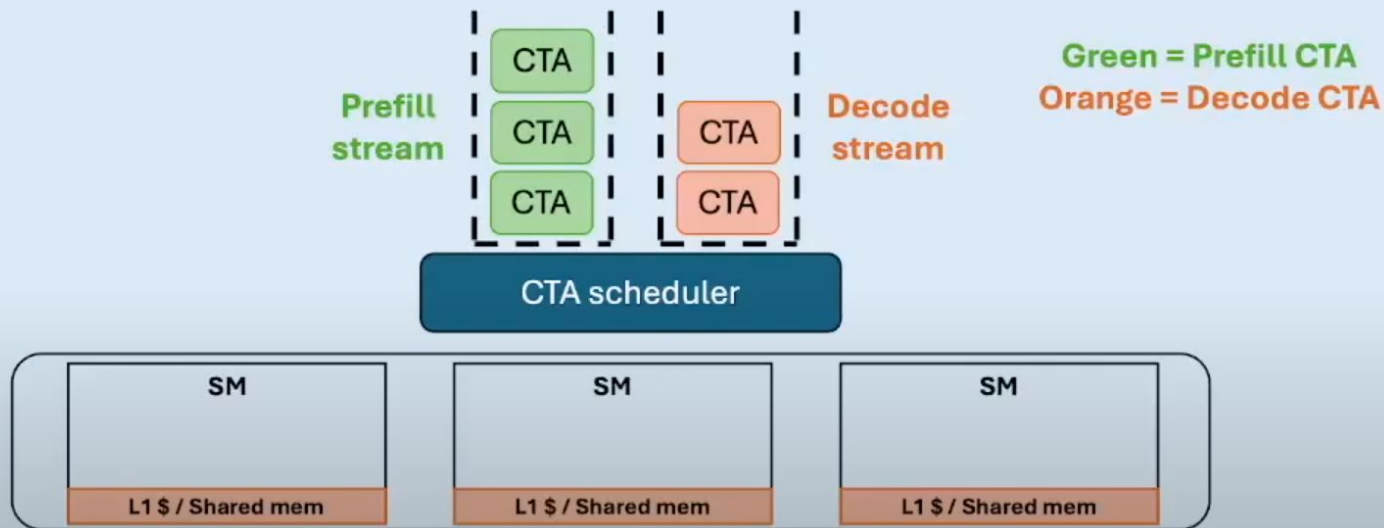
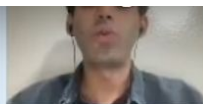
Introducing POD-Attention!



- **POD-Attention:** Combines prefills and decodes into a single kernel with guaranteed SM co-location.
- Key idea: **SM-aware CTA scheduling**
 - Guarantees each SM runs prefill and decode CTAs in parallel.
 - Enables the CTA scheduler to overlap the two operations.
 - Utilizes compute and memory bandwidth simultaneously.

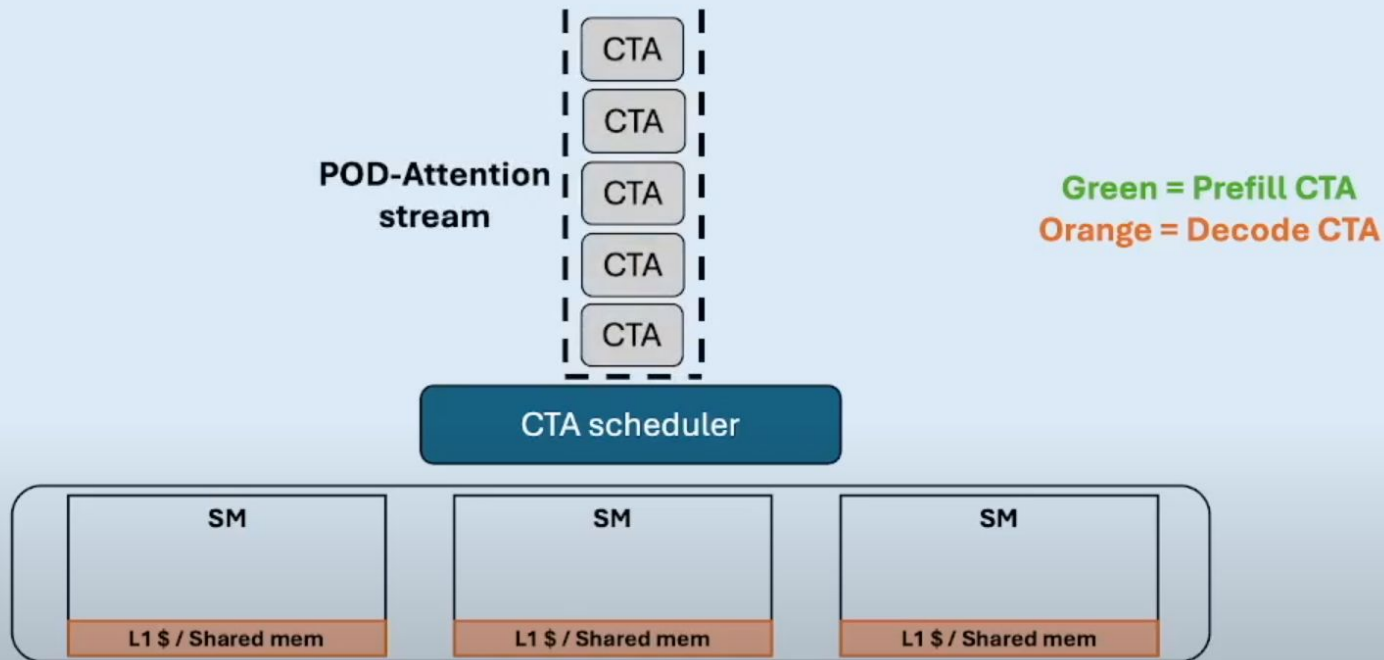
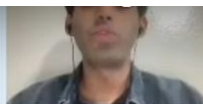


POD-Attention: Fused kernels





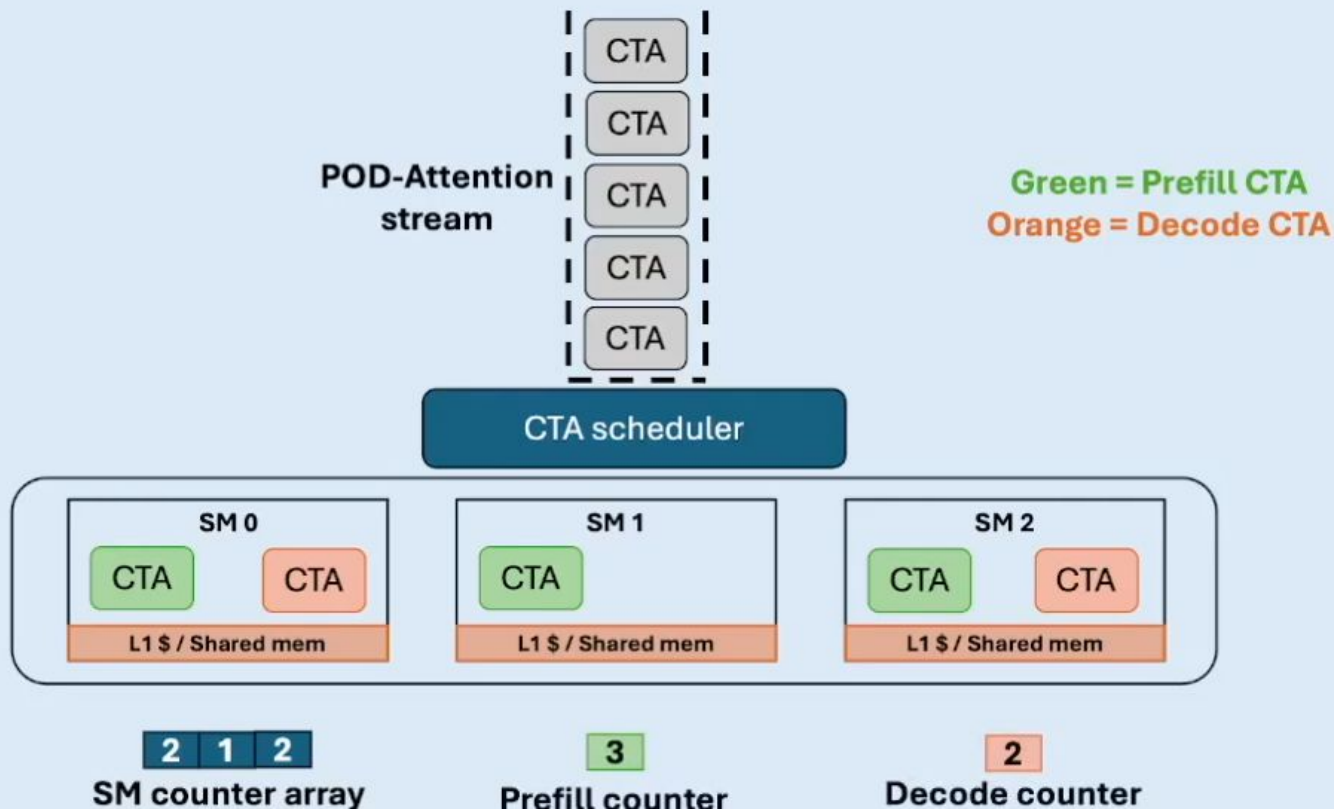
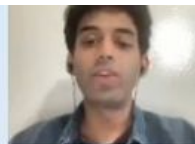
POD-Attention: Fused kernels



We create a new kernel with enough CTAs for both prefill and decode kernels.

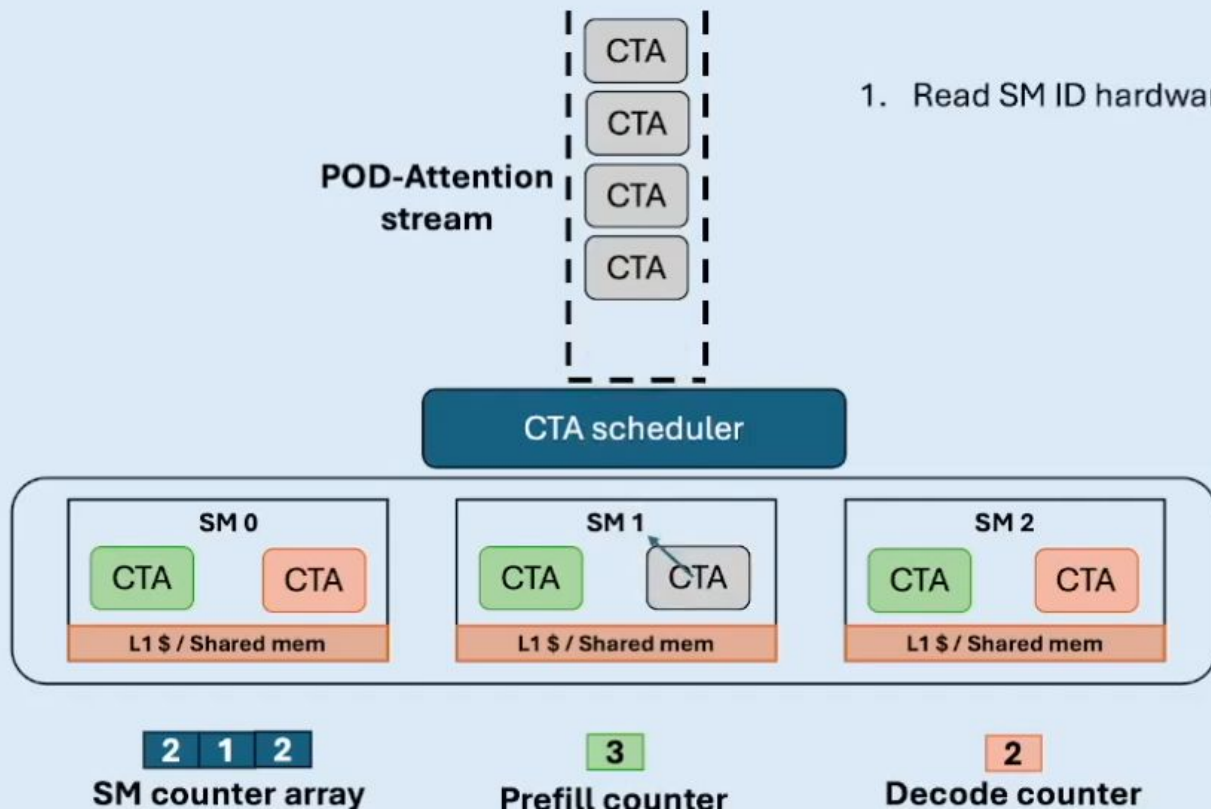
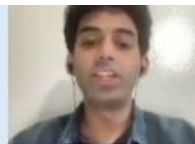


POD-Attention: SM-aware CTA scheduling



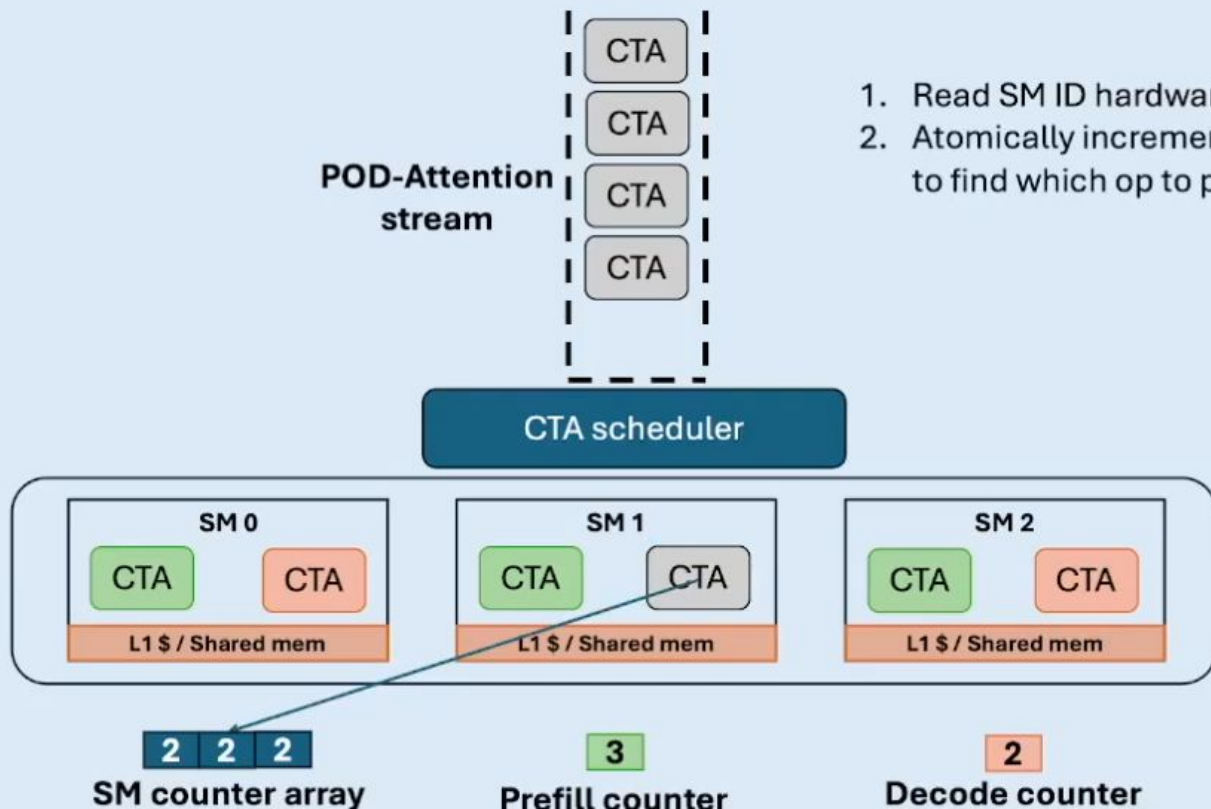
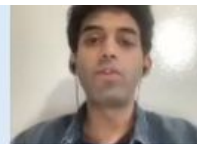


POD-Attention: SM-aware CTA scheduling



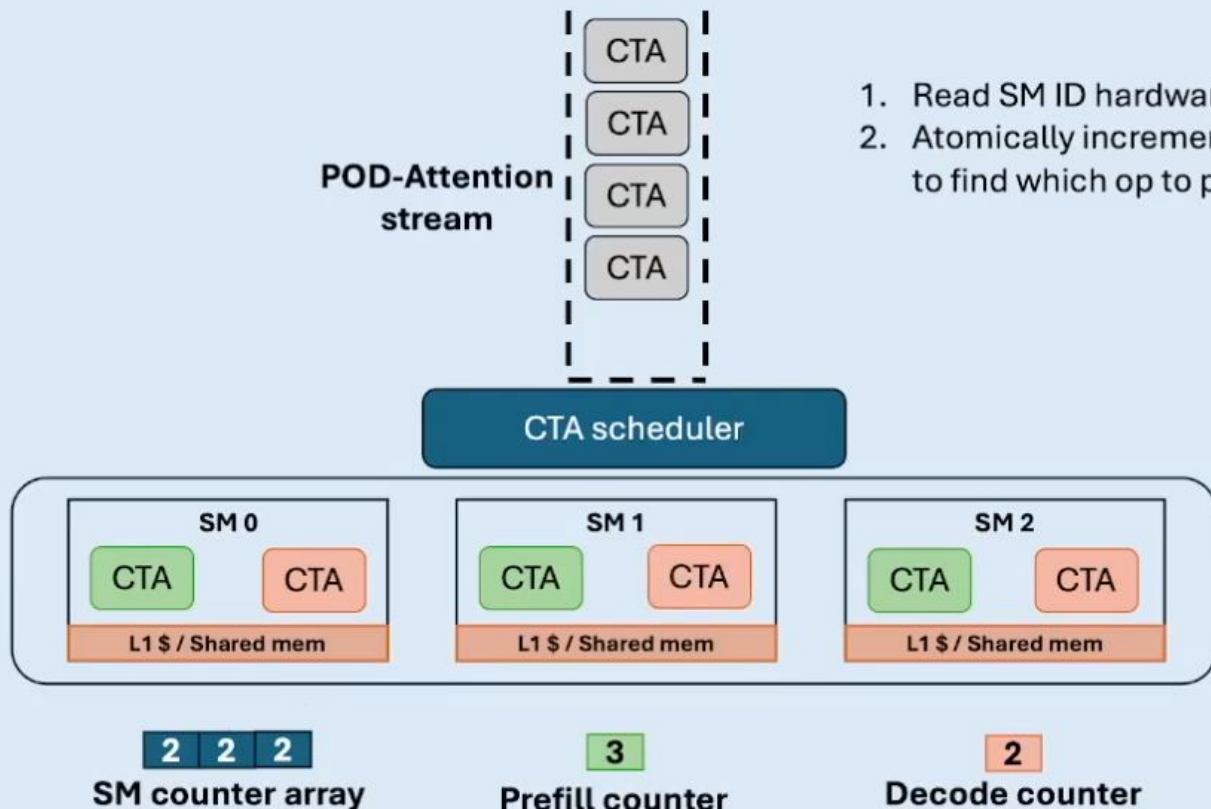
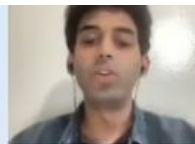


POD-Attention: SM-aware CTA scheduling



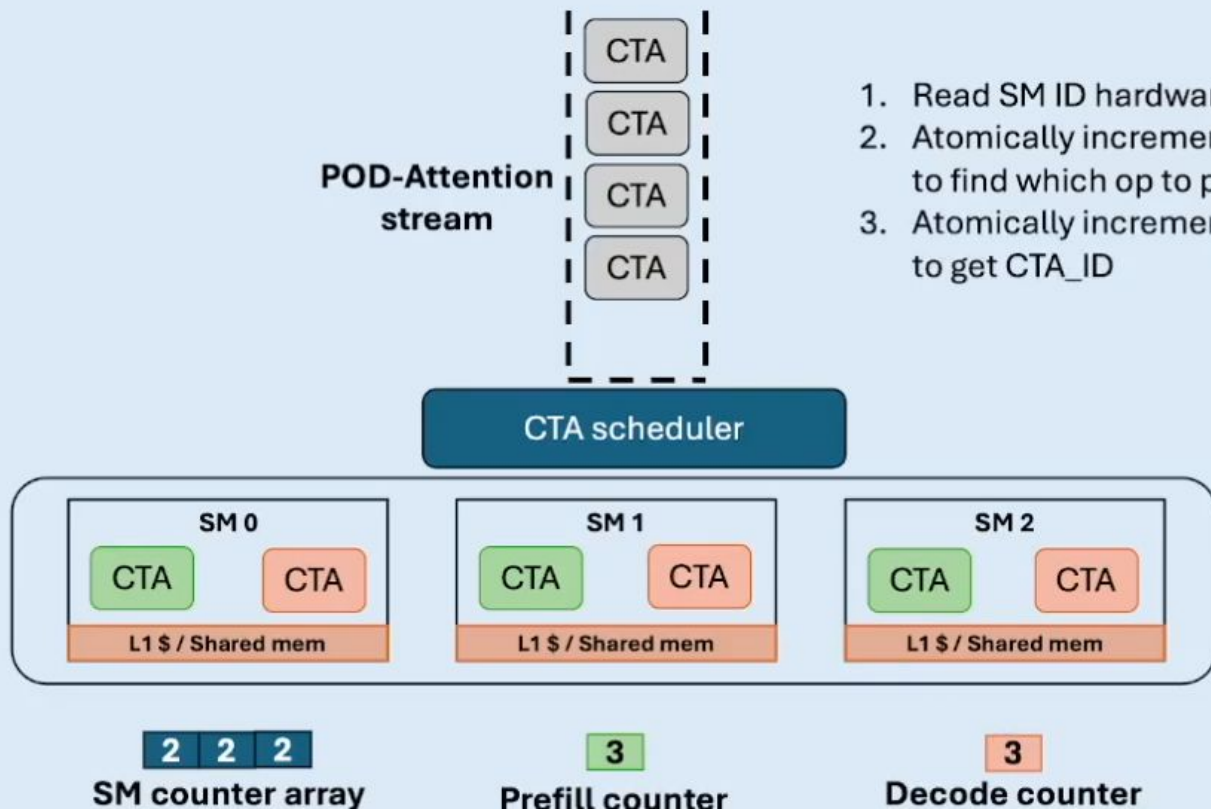
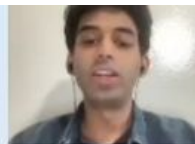


POD-Attention: SM-aware CTA scheduling





POD-Attention: SM-aware CTA scheduling



```

if (threadIdx.x == 0) {
    // Only the first thread in the CTA (Cooperative Thread Array) performs the scheduling work
    // Only the thread 0 in the thread block decides which operation (Prefill or Decode) will be run
    // and which CTA will handle it.
    int sm_id;
    // Variable to store the Streaming Multiprocessor (SM) ID that this CTA is running on.
    asm volatile("mov.u32 %0, %%smid;" : "=r"(sm_id));
    // Inline PTX assembly code embedded in CUDA: Move the current SM ID into sm_id
    // asm tells the compiler that we are using inline assembly.
    // volatile prevents the compiler from optimizing away this instruction, ensuring that this always
    // executes as written.
    // mov.u32 %0, %%smid is a PTX instruction, that moves the value of the special register %%smid into
    // the operand %0.
    // %%smid: A special register in PTX that holds the ID of the SM on which the thread is executing.
    // %0: A placeholder for the output operand (which is mapped to a C++ variable).
    // The colon : separates the assembly code from the operands.
    // "=r"(sm_id) tells that result of the inline assembly should be stored in a general-purpose
    // register (r) and assigned to the C++ variable sm_id.
    // The = indicates that sm_id is an output-only operand.
    const int ratio = (prefill_ratio + decode_ratio);
    // Determine the ratio of prefill vs decode work
    // The total "weight" of operations, used for probabilistic selection
    int op, ticket = (atomicAdd(&sm_ctr[sm_id], 1) % ratio);
    // Atomically increment the counter for this SM and take modulo with ratio to decide operation type
    // atomicAdd returns the previous value of sm_ctr[sm_id] before the increment.
    // Ticket cycles between 0 and ratio - 1.
    // op is just declared, not assigned. ticket holds the required value.

```

```

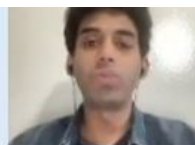
if (ticket < prefill_ratio)
    op = PREFILL;
else
    op = DECODE;
// Decide operation type based on the ticket
// If the ticket is less than prefill_ratio, it is assigned prefill task otherwise decode task.
int cta_id = atomicAdd(&cta_assign[op], 1);
// Get the next CTA ID to process for the chosen operation
// We would assign this particular CTA ID the task of decode or Prefill.
// This is because single SM contains multiple CTAs. So, we need to assign accordingly.
// Check if CTA ID exceeds the max allowed for that operation
if (op == PREFILL && cta_id >= prefill_ctas) {
    // Case 1: We intended to assign a PREFILL operation,
    // but the assigned CTA ID exceeds the maximum number of allowed prefill CTAs.
    op = DECODE;
    // Fallback: switch the operation to DECODE instead.
    cta_id = atomicAdd(&cta_assign[op], 1);
    // Reassign a new CTA ID specifically for the DECODE operation.
    // This ensures the new CTA ID is valid and within decode limits.
}
else if (op == DECODE && cta_id >= decode_ctas) {
    // Case 2: We intended to assign a DECODE operation,
    // but the assigned CTA ID exceeds the maximum number of allowed decode CTAs.
    op = PREFILL;
    // Fallback: switch the operation to PREFILL instead.
    cta_id = atomicAdd(&cta_assign[op], 1);
    // Reassign a new CTA ID specifically for the PREFILL operation.
}
// Save assigned CTA ID and operation type to shared memory for other threads to access
shared_mem[0] = cta_id;
shared_mem[1] = op;
}
__syncthreads();
// Synchronize all threads in the CTA to ensure the leader thread finished scheduling
int cta_id = shared_mem[0];
const int op = shared_mem[1];

```

The main code snippet for SM-Aware CTA Scheduling



Attention-specific Optimizations



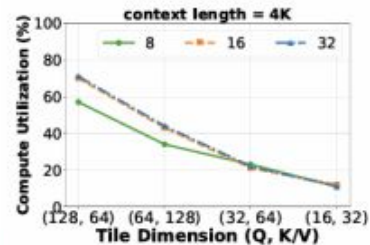
- Decode tiling
 - Smaller tiles minimize compute utilization of decode CTAs
- Varying number of CTAs per SM
 - Enables fine-grained overlap of prefill and decode CTAs
- Limited prefill splitting
 - Reduces memory bandwidth usage of prefill CTAs

Attention Specific Optimizations:

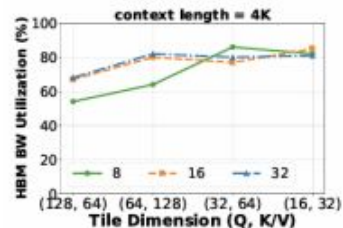
1. Tile Sizes: (Make Effective use of Tensor Cores)

Data tiling is crucial for making efficient use of tensor cores, which offer around $8\times$ more throughput than CUDA cores, while also improving shared memory usage. However, its benefits vary depending on the operation. In decode operations, which process a single token per request, the tile length in the Query Sequence Length (QSL) dimension is just 1. In Group Query Attention, the QSL tile length increases slightly (typically between 2 and 8), but data reuse remains insignificant, leaving performance limited by memory bandwidth.

Flash Attention tackles this by using larger tile sizes (64–128) for the QSL dimension, which boosts compute utilization up to 70% for large tiles but introduces redundant computation due to zero padding. Despite this, decode performance stays memory-bound, so large tiles help saturate memory bandwidth without harming performance. However, in fused kernels where decode and prefill run together, large decode tiles waste compute resources and hurt prefill performance, since tensor cores are shared. To avoid this, a smaller QSL tile length of 16 is used (the minimum for CUTLASS on A100 GPUs), reducing decode compute utilization to about 10% and allowing prefill to better use tensor cores - without impacting decode performance, especially at large batch sizes.



(a) Compute utilization.



(b) DRAM BW utilization.

Impact of Decode Tile Size on Compute and Mem BW Utilization.

Attention Specific Optimizations:

2. Concurrent CTAs per SM: (More opportunities for fine grained scheduling and co-location)

Keeping more CTAs per SM allows us to allocate CTAs to prefill and decode in different proportions depending on Batch composition.

Prefills benefit from fewer CTAs per SM as it allows each CTA to access more shared memory, enabling use of larger Tile sizes.

On the other hand, Decodes do not benefit from larger tile sizes and therefore using more CTAs per SM can be beneficial Since it allows fine grained scheduling.

Therefore, POD Attention allows 2 modes: 2 CTAs Per SM and 4 CTAs per SM, adjusts automatically at runtime.

3. Virtual Decode CTAs (Avoid Over-allocating Shared Memory to Decodes)

As decode uses smaller tile sizes, the memory requirement of decode is a quarter of the prefill requirement. To avoid Over-allocating shared memory to decodes, we divide each decode CTA into virtual CTAs. If original decode CTA has 4 warps, each virtual CTA contains one warp which uses a quarter of the shared memory of original CTA. This way, the sum of shared memory used by all virtual CTAs in a regular CTA will be close to shared memory used by Prefills.

Hence, virtual decode CTAs balance the shared memory used by prefill and decode.

Attention Specific Optimizations:

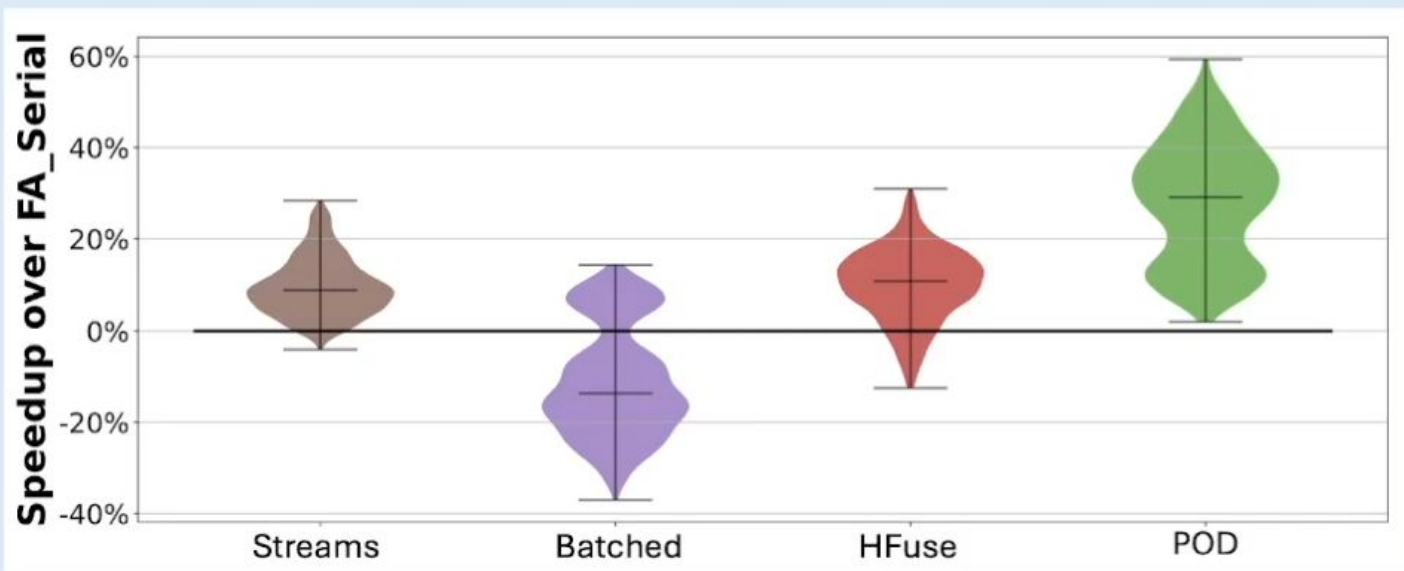
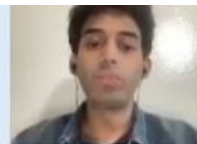
4. Limiting Prefill Splits:

Flash Attention parallelizes computation across query heads and QSL tiles, while FlashDecoding further splits work across the key/value (K/V) dimension when needed to keep the GPU busy, especially during decoding where QSL is 1. This extra splitting increases memory bandwidth usage, as multiple thread blocks fetch the same query data independently. For chunked-prefills, which process a limited number of tokens per iteration, FlashAttention applies this splitting to boost parallelism and speed up computation, with little downside in a prefill-only kernel.

However, in a fused kernel combining prefill and decode, too many splits cause memory bandwidth contention between prefill and decode, reducing efficiency. To avoid this, the number of splits is carefully limited (empirically set to fill at most two full waves of CTAs), balancing parallelism without hurting decode performance.



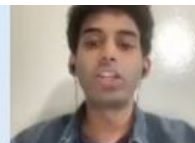
POD-Attention Performance



POD beats all existing alternatives, with up to 61% speedup (33% average).



LLM Evaluation: Baselines and Workloads



Inference frameworks

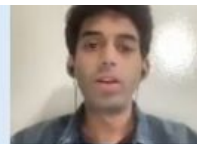
Framework	Description
vLLM [SOSP '23]	Popular framework that prioritizes prefill latency (TTFT).
Sarathi [OSDI '24]	Uses hybrid-batching to improve decode latency (TBT).
Sarathi + POD	Sarathi enhanced with POD-Attention.

Models and hardware

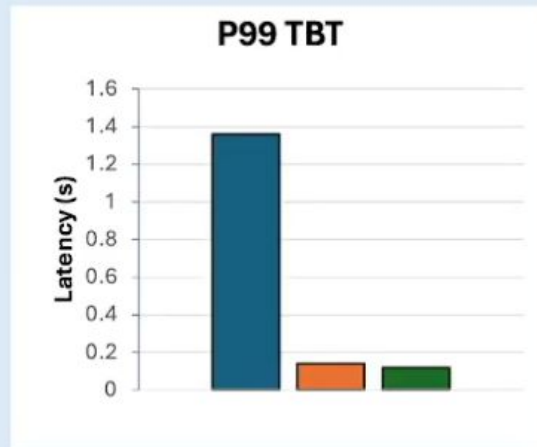
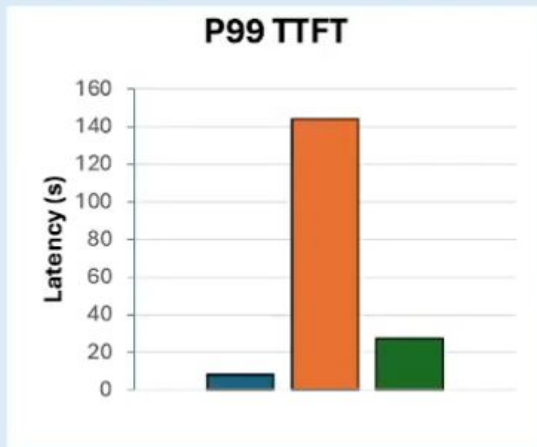
Model	GPU	# Q Heads	# KV Heads	# Layers
Yi-6B	1 A100	32	4	32
Llama-2-7B	2 A100s	32	32	32
Llama-3-8B	2 A100s	32	8	32



Online Inference Performance



Llama 3 8B, arXiv-Summarization, QPS = 0.95



■ vLLM ■ Sarathi ■ Sarathi + POD

Lower is better

- vLLM provides low TTFT but **high TBT**, stalling nearly all requests
- Sarathi provides low TBT, but **high TTFT**
- Sarathi + POD reduces TBT stalls with TTFT closer to vLLM